

# Claude Certified Architect — Foundations Certification

## مطالعہ گائیڈ (سرکاری امتحانی گائیڈ کی بنیاد پر)

### تعارف

سرٹیفیکیشن اس بات کی تصدیق کرتی ہے کہ ایک ماہر حقیقی Claude Certified Architect — Foundations Claude Code، Claude Agent SDK، Claude API، اور Model Context Protocol (MCP) — یعنی بنیادی علم کا جائزہ لیتا ہے — کے ساتھ پروڈکشن ایپلیکیشنز بنائی جاتی ہیں۔ Claude و بنیادی ٹیکنالوجیز جن سے

agentic systems امتحانی سوالات حقیقت پسندانہ صنعتی منظرناموں پر مبنی ہوتے ہیں: کسٹمر سپورٹ کے لیے، multi-agent research pipelines، ڈیزائن کرنا، Claude Code کو CI/CD میں ضم کرنا، developer productivity، structured data بنانا، اور غیر ساختہ دستاویزات سے tools نکالنا۔

### هدف امیدوار

کرتا ہے آپ ship کے ساتھ پروڈکشن ایپلیکیشنز ڈیزائن اور Claude کے جو solution architect مثالی امیدوار ایک کے پاس کم از کم 6 ماہ کا عملی تجربہ ہونا چاہیے:

- Claude Agent SDK** — multi-agent orchestration، subagents کو delegate کرنا، tool integration، lifecycle hooks
- Claude Code** — CLAUDE.md، MCP servers، Agent Skills، planning mode
- Model Context Protocol (MCP)** — backend integration کے لیے tools اور resources
- Prompt engineering** — JSON schemas، few-shot examples، data extraction templates
- Context windows** — لمبی دستاویزات کے ساتھ کام، multi-agent context passing
- CI/CD pipelines** — automated code review، test generation
- Escalation and reliability** — error handling، human-in-the-loop

### امتحانی فارمیٹ

| پیرامیٹر         | قدر                                      |
|------------------|------------------------------------------|
| سوال کی قسم      | Multiple choice (میں سے 1 درست 4)        |
| اسکورنگ          | 100–1000 scale، passing score <b>720</b> |
| Guessing penalty | نہیں (سوال کا جواب دیں!)                 |
| Scenarios        | 8 4 (randomly selected) میں سے           |

## امتحانی مواد: 5 ڈومینز

| ڈومین                                       | وزن |
|---------------------------------------------|-----|
| 1. Agent architecture and orchestration     | 27% |
| 2. Tool design and MCP integration          | 18% |
| 3. Claude Code configuration and workflows  | 20% |
| 4. Prompt engineering and structured output | 20% |
| 5. Context management and reliability       | 15% |

## امتحانی منظر نامہ

### Scenario 1: Customer Support Agent

returns, billing disputes, اور account issues کے استعمال کرتے ہوئے Claude Agent SDK بناتے ہیں جو آپ ایک agent MCP tools (get\_customer, lookup\_order, process\_refund, escalate\_to\_human) agent سنبھالتا ہے۔ مناسب first-contact resolution +% استعمال کرنا 80 دفعہ (escalation کے ساتھ، مناسب first-contact resolution +% استعمال کرنا 80 دفعہ) کے ساتھ،

### Scenario 2: Claude Code کے ساتھ Code Generation

code generation, refactoring, تیز کرنے کے لیے استعمال کرتے ہیں development کو Claude Code آپ کے ساتھ CLAUDE.md configuration اور custom slash commands آپ کو اسدے debugging, documentation کے ساتھ integrate کب استعمال کرنا planning mode کرنا، اور یہ سمجھنا کے

### Scenario 3: Multi-Agent Research System

tasks کو coordinator agent specialized subagents سونپتا: web research, document analysis, synthesis, تیار کرنے کے لیے چاہئیں reports کے ساتھ مکمل citations کے سسٹم کو report generation اور

### Scenario 4: Developer Productivity Tools

agent engineers کو unfamiliar codebases explore کرنے، boilerplate code اور routine tasks automate MCP servers اور Built-in tools (Read, Write, Bash, Grep, Glob) میں مدد دیتا ہے۔ کیے جاتے ہیں

### Scenario 5: Claude Code for Continuous Integration

pull اور automated code reviews, test generation, Claude Code کو CI/CD pipeline تاکہ request feedback کم سے کم false positives ڈیزائن ہونے چاہئیں کے Prompts حاصل ہو

## Scenario 6: Structured Data Extraction

کرتا ہے، validate کے ذریعے JSON schemas کو output، سسٹم غیر ساختہ دستاویزات سے معلومات نکالتا ہے اور high accuracy کے لیے edge cases پر سنبھالنا چاہیے۔ اس کا برقرار رکھنا اس کے لیے ضروری ہے۔

## Scenario 7: Conversational AI Architecture Patterns

multi-turn conversational systems میں جن میں context window management، turns متعدد، memory strategies، tool design کے لیے execution محفوظ، کا برقرار رکھنا instructions میں اور مہم یا متضاد، user inputs کو سنبھالنا شامل ہے۔

## Scenario 8: Agentic AI Tools (ہماری مدد کریں — مواد موجود نہیں)

میں شامل نہیں ہے۔ guide کے لیے دیے گئے لیکن ابھی تک اس کے candidates کی اطلاع امتحان دینے والے scenario اس کے لیے share میں GitHub Issues کے سوالات دیکھیں، تو ان میں سے اگر آپ نے اصل امتحان میں اس کے لیے شامل کر سکیں۔ آپ کا تعاون اس شخص کی مدد کرے گا جو امتحان کی تیاری کے لیے coverage تاکہ اس کے لیے مکمل رہے۔

## سرکاری دستاویزات

| وسیلہ                                       | URL                                                                                                                                                       |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Claude API — Messages                       | <a href="https://platform.claude.com/docs/en/api/messages">https://platform.claude.com/docs/en/api/messages</a>                                           |
| Claude API — Tool Use                       | <a href="https://platform.claude.com/docs/en/build-with-claude/tool-use">https://platform.claude.com/docs/en/build-with-claude/tool-use</a>               |
| Claude API — Message Batches                | <a href="https://platform.claude.com/docs/en/build-with-claude/message-batches">https://platform.claude.com/docs/en/build-with-claude/message-batches</a> |
| Claude Agent SDK — Overview                 | <a href="https://platform.claude.com/docs/en/agent-sdk/overview">https://platform.claude.com/docs/en/agent-sdk/overview</a>                               |
| Claude Agent SDK — Hooks                    | <a href="https://platform.claude.com/docs/en/agent-sdk/hooks">https://platform.claude.com/docs/en/agent-sdk/hooks</a>                                     |
| Claude Agent SDK — Subagents                | <a href="https://platform.claude.com/docs/en/agent-sdk/subagents">https://platform.claude.com/docs/en/agent-sdk/subagents</a>                             |
| Claude Agent SDK — Sessions                 | <a href="https://platform.claude.com/docs/en/agent-sdk/sessions">https://platform.claude.com/docs/en/agent-sdk/sessions</a>                               |
| Model Context Protocol (MCP)                | <a href="https://modelcontextprotocol.io/">https://modelcontextprotocol.io/</a>                                                                           |
| MCP — Tools                                 | <a href="https://modelcontextprotocol.io/docs/concepts/tools">https://modelcontextprotocol.io/docs/concepts/tools</a>                                     |
| MCP — Resources                             | <a href="https://modelcontextprotocol.io/docs/concepts/resources">https://modelcontextprotocol.io/docs/concepts/resources</a>                             |
| MCP — Servers                               | <a href="https://modelcontextprotocol.io/docs/concepts/servers">https://modelcontextprotocol.io/docs/concepts/servers</a>                                 |
| Claude Code — Documentation                 | <a href="https://code.claude.com/docs/en/overview">https://code.claude.com/docs/en/overview</a>                                                           |
| Claude Code — CLAUDE.md and Memory          | <a href="https://code.claude.com/docs/en/memory">https://code.claude.com/docs/en/memory</a>                                                               |
| Claude Code — Skills (incl. slash commands) | <a href="https://code.claude.com/docs/en/skills">https://code.claude.com/docs/en/skills</a>                                                               |

|                                               |                                                                                                                                                                                   |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Claude Code — Hooks                           | <a href="https://code.claude.com/docs/en/hooks">https://code.claude.com/docs/en/hooks</a>                                                                                         |
| Claude Code — Sub-agents                      | <a href="https://code.claude.com/docs/en/sub-agents">https://code.claude.com/docs/en/sub-agents</a>                                                                               |
| Claude Code — MCP Integration                 | <a href="https://code.claude.com/docs/en/mcp">https://code.claude.com/docs/en/mcp</a>                                                                                             |
| Claude Code — GitHub Actions CI/CD            | <a href="https://code.claude.com/docs/en/github-actions">https://code.claude.com/docs/en/github-actions</a>                                                                       |
| Claude Code — GitLab CI/CD                    | <a href="https://code.claude.com/docs/en/gitlab-ci-cd">https://code.claude.com/docs/en/gitlab-ci-cd</a>                                                                           |
| Claude Code — Headless (non-interactive mode) | <a href="https://code.claude.com/docs/en/headless">https://code.claude.com/docs/en/headless</a>                                                                                   |
| Prompt Engineering Guide                      | <a href="https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview">https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview</a> |
| Extended Thinking                             | <a href="https://platform.claude.com/docs/en/build-with-claude/extended-thinking">https://platform.claude.com/docs/en/build-with-claude/extended-thinking</a>                     |
| Anthropic Cookbook (code examples)            | <a href="https://github.com/anthropics/anthropic-cookbook">https://github.com/anthropics/anthropic-cookbook</a>                                                                   |

## نظریاتی بنیادیں: حصہ ۱

یہ حصہ تمام نظریاتی احاطہ کرتا ہے جس کی آپ کو امتحان میں کامیابی کے لیے ضرورت ہے۔ مواد کو کے مطابق — اس سے آپ کے domains کے مطابق ترتیب دیا گیا ہے، نہ کہ امتحانی concepts ٹیکنالوجیز اور موضوع کی زیادہ گہری سمجھ پیدا کر سکتے ہیں۔

## کی بنیادیں Claude API — Model Interaction: باب 1

دستاویزات: [Messages API](#) | [Prompt Engineering](#)

### 1.1 API Request Structure

Claude API کی request کی Claude Messages API کی پیروی کرتا ہے request-response model ایک شامل ہوتا ہے:

```
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 1024,
  "system": "You are a helpful assistant.",
  "messages": [
    {"role": "user", "content": "Hi!"},
    {"role": "assistant", "content": "Hello!"},
    {"role": "user", "content": "How are you?"}
  ],
  "tools": [...],
  "tool_choice": {"type": "auto"}
}
```

### م فیلڈر:

- `model` — model selection ( `claude-opus-4-6` , `claude-sonnet-4-6` , `claude-haiku-4-5` )
- `max_tokens` — response کی تعداد میں زیادہ سے زیادہ
- `system` — system prompt (model کی تعریف کرتا ہے)
- `messages` — conversation history (تاریخچه گفتگو) (آپ کو مکمل گفتگو)
- `tools` — tools کی definitions دستیاب
- `tool_choice` — tool selection strategy

## 1.2 Message Roles

`messages` array میں roles استعمال کرتا ہے:

- `user` — user messages
- `assistant` — model responses (history میں شامل کیا جاتا ہے)
- `tool` — tool call results (role واضح طور پر `tool_result` میں کیا جاتا ہے؛ `set` میں ظاہر ہوتا ہے)

Model requests بھیجی جائے گی۔ `conversation history` میں آپ کو مکمل API request انتہائی اہم: ہر آزاد ہوتا ہے call محفوظ نہیں رکھتا — ہر state درمیان

## 1.3 Response میں `stop_reason` Field

کیوں روکی generation model شامل ہوتا ہے، جو بتاتا ہے کہ `stop_reason` میں Claude API response

| Value           | Description                   | Action                                             |
|-----------------|-------------------------------|----------------------------------------------------|
| "end_turn"      | Model نے response مکمل کر لیا | user کو دکھائیں نتیجہ                              |
| "tool_use"      | Model tool call کرنا چاہتا ہے | Tool execute کریں اور result دیں                   |
| "max_tokens"    | Token limit ختم ہو گئی        | Response truncated ہو سکتا ہے؛ limit بڑھانی پڑے گی |
| "stop_sequence" | Stop sequence مل گئی          | کریں handle کہ مطابق application logic اپنی        |

کو کنٹرول کرتے agent loop سب سے اہم ہیں — یہی "end\_turn" اور "tool\_use" میں Agentic systems ہیں۔

## 1.4 System Prompt

System prompt متعین کرتا ہے behavioral rules اور context جو instruction ایک خاص

- messages array الگ ہوتا؛ یہ system field کا حصہ نہیں ہوتا؛ بلکہ الگ array
- user messages پر فوقیت رکھتا ہے
- میں لاگو رہتا ہے conversation ہوتا ہے اور پوری load ایک بار
- کی تعریف کے لیے استعمال ہوتا ہے output format اور role, constraints,

پیدا کر سکتی ہے مثال کے طور پر tool associations غیر ارادی wording کی system prompt: امتحان کے لیے اہم کو ضرورت سے زیادہ get\_customer کو model کی تصدیق کریں "جیسی ہدایت customer پر،" ہمیشہ استعمال کرنے پر مجبور کر سکتی ہے، حتیٰ کے جب یہ ضروری نہ ہو

## 1.5 Context Window

Context window text (tokens) جسے اس میں process ایک وقت میں model کی کل مقدار ہے جسے شامل ہے:

- System prompt
- message history مکمل
- Tool definitions
- Tool results

مسائل context-window:

1. **Lost-in-the-middle effect:** models input سے اعتماد سے زیادہ موجود معلومات پر اختتام پر موجود معلومات کو زیادہ input سے اعتماد سے زیادہ models process کر سکتے ہیں۔ حل: اہم معلومات شروع یا آخر کے قریب miss کرتے ہیں، مگر درمیان کی تفصیلات process رکھیں
2. **Tool results** واپس کر کے 40+ fields tool شامل کرتا ہے اگر output میں tool call context کا جمع ہونا: ہر Tool results کا بڑا حصہ ضائع ہو جاتا ہے context مگر صرف 5 اہم ہوں، تو
3. **Progressive summarization:** history compress کرتے وقت numeric values, percentages, اور dates اکثر گم ہو کر غیر واضح ہو جاتے ہیں ("تقریباً"، "کچھ"، "چند")

## 2 Tools اور tool\_use: باب

Tool Use: دستاویزات

## 2.1 tool\_use کیا

برا Model کی اجازت دیتا external functions call Claude جو mechanism ایک tool\_use result کرتا اور execute اس code بناتا؛ آپ کا structured tool call request میں چلاتا — و code راست واپس کرتا

## 2.2 Tool Definition

کیا جاتا define کے ذریعہ JSON schema ایک tool

```
{
  "name": "get_customer",
  "description": "Finds a customer by email or ID. Returns the customer profile, including name, email, order history, and account status. Use this tool BEFORE lookup_order to verify the customer's identity. Accepts an email (format: user@domain.com) or a numeric customer_id.",
  "input_schema": {
    "type": "object",
    "properties": {
      "email": {"type": "string", "description": "Customer email"},
      "customer_id": {"type": "integer", "description": "Numeric customer ID"}
    },
    "required": []
  }
}
```

کے انتہائی اہم ہلو Tool description:

- Description tool selection** کی بنیاد پر descriptions کو ان کی LLM tools mechanism کا بنیادی Minimal descriptions ("Customer information retrieve کرتا) اُن مواقع پر غلطیوں کا (کرتا) overlap کے دو سر کے tools باعث بنتی ہیں جب کرتے ہوں
- Description میں شامل کریں:**
  - کیا کرتا اور کیا واپس کرتا Tool
  - Input formats اور example values
  - Edge cases اور constraints
  - مقابلہ میں کب استعمال و tool similar alternatives
- کی analyze\_document اور analyze\_content سے بچیں اگر overlapping descriptions ایک جیسے یا انہیں خلط ملط کر دے گا model تقریباً ایک جیسی ہوں، تو descriptions
- Built-in tools** MCP tools: agents similar functionality والے built-in tools (Read, Grep) کو MCP tools — مضبوط بنائیں MCP tool descriptions پر ترجیح دے سکتے ہیں اس سے بچنے کے لیے concrete advantages, unique data, یا context جو built-in tools میں کر سکتے ہیں

## 2.3 tool\_choice Parameter

کیسے منتخب کرتا model tools کے کنٹرول کرتا tool\_choice

| Value                                                       | Behavior                                                      | کب استعمال کریں                                    |
|-------------------------------------------------------------|---------------------------------------------------------------|----------------------------------------------------|
| <code>{ "type": "auto" }</code>                             | Model کرنا tool call خود ط کرتا یا کرنا text یا میں جواب دینا | default زیاد تر حالات میں                          |
| <code>{ "type": "any" }</code>                              | Model کرنا tool call کو لازماً کوئی                           | guaranteed جب آپ کو structured output چاہیں        |
| <code>{ "type": "tool", "name": "extract_metadata" }</code> | Model کرنا tool call کو لازماً ایک مخصوص tool کو              | forced first step / execution order جب آپ کو چاہیں |

ا: م منظر نامہ:

- `tool_choice: "any"` + multiple extraction tools → model منتخب کرتا ہے، مگر پھر بھی tool بہترین model ملنا structured output
- Forced selection → enrichment (مثلاً) کی ضمانت چاہیں action جب آپ کو کسی خاص پم ( `extract_metadata` )

## 2.4 Structured Output JSON Schemas

حاصل کرنے کا سب سے قابلِ structured output Claude استعمال کرنا `tool_use` کے ساتھ JSON schemas ی: اعتماد طریقہ

- Syntax درست ہے (trailing commas، غائب نہیں ہوتے braces) کی ضمانت دیتا ہے JSON کے اعتبار سے درست ہے (ہوتے)
- (موجود ہوتے ہیں required fields) نافذ کرتا ہے structure مطلوب ہے
- Semantic correctness (پھر بھی غلط ہو سکتی ہے values) کی ضمانت نہیں دیتا

Schema design — بنیادی اصول:



```
{
  "type": "object",
  "properties": {
    "category": {
      "type": "string",
      "enum": ["bug", "feature", "docs", "unclear", "other"]
    },
    "category_detail": {
      "type": ["string", "null"],
      "description": "Details if category = 'other' or 'unclear'"
    },
    "severity": {
      "type": "string",
      "enum": ["critical", "high", "medium", "low"]
    },
    "confidence": {
      "type": "number",
      "minimum": 0,
      "maximum": 1
    },
    "optional_field": {
      "type": ["string", "null"],
      "description": "Null if the information was not found in the source"
    }
  },
  "required": ["category", "severity"]
}
```

**Schema design rules:**

1. **Required** □ **optional:** □ **میشد** □ دستیاب □ **required** □ **fields** کو صرف اُن □ **مقیاب** □ **Required fields** model پر data کو □ **values** □ **غائب** □ **ون** □ **پر** □ **مجبور** کر سکتی □ **یں** □ **گھڑن** □
2. **Nullable fields:** □ ایسی معلومات □ **لی** □ **"type": ["string", "null"]** □ **استعمال** کریں جو ممکن □ **طور** □ **پر** □ **بجائ** □ **اس** □ **ک** □ **و** □ **خود** □ **ساخت** □ **null** □ **Model** □ **موجود** □ **ن** □ **وں** □ **بنا** □ **کر** □ **سکتا** □ **و** □ **value** □ **واپس** □ **کر** □ **سکتا** □ **ہو** □ **نہ** □ **نہ** □ **وں** □ **سد** □ **بجائ** □ **ک** □ **لی** □ **predefined categories** □ **اپنی** □ **Enums with "other":** □ **شامل** □ **کریں** □ **string** □ **ایک** □ **+** □ **"other"** □
3. **Enums with "other":** □ **شامل** □ **کریں** □ **string** □ **ایک** □ **+** □ **"other"** □
4. **Enum "unclear":** □ **بار** □ **میں** □ **پُر** □ **اعتماد** □ **نہ** □ **و** □ **ایماندار** □ **model category** □ **اُن** □ **صورتوں** □ **ک** □ **لی** □ **جب** □ **سد** □ **بتر** □ **□□□** □ **category** □ **غلط** □ **"unclear"** □

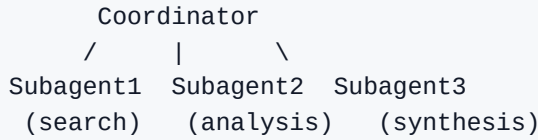
## 2.5 Syntax □ بمقابلہ Semantic Errors

| Error type | Example                                                | Mitigation                                                 |
|------------|--------------------------------------------------------|------------------------------------------------------------|
| Syntax     | Invalid JSON، field غلط type                           | JSON schema کے ساتھ tool_use (ختم کر دینا) (               |
| Semantic   | Totals match نہیں کرتے، value field غلط، hallucination | Validation checks، feedback کے ساتھ retry، self-correction |



### 3.3 Hub-and-Spoke: Coordinator اور Subagents

Multi-agent architecture عموماً hub-and-spoke topology میں بنائی جاتی ہے:



Coordinator کی ذمہ داریاں:

- Task کو subtasks میں تقسیم کرنا
- (dynamic selection) درکار subagents میں سے طے کرنا کہ کون سا
- کو سونپنا subagents کام
- کرنا validate نتائج جمع اور
- Errors اور retries handle کرنا
- تک پہنچانا user نتائج

الگ ہوتا context subagents کا: اصول

- Subagents conversation history کی coordinator خود بخود inherit نہیں کرتے
- میں واضح طور پر دینا ہوتا context subagent prompt تمام ضروری
- Subagents memory share کے درمیان calls نہیں کرتے
- (observability اور error control کے ذریعے گزرتی ہے coordinator communication تمام)

### 3.4 Subagents کے لیے Task Tool بنانا

Subagents Task tool کے ذریعے spawn میں لے جاتے ہیں:

```
# Coordinator میں allowedTools کے "Task" ہونا چاہیے
coordinator_agent = AgentDefinition(
    allowed_tools=["Task", "get_customer"]
)
```

Explicit context passing لازمی ہے:

```
# Bad: subagent کے پاس context نہیں ہے
Task: "Analyze the document"

# Good: context prompt میں مکمل
Task: "Analyze the following document.
Document: [full document text]
Prior search results: [web search results]
Output format requirements: [schema]"
```

**Parallel spawning:** coordinator ایک response میں متعدد Task s call کر سکتا ہے — subagents parallel چلتے ہیں:

```
# Coordinator کے ایک response میں شامل ہے:
Task 1: "Search for articles about X"
Task 2: "Analyze document Y"
Task 3: "Search for articles about Z"
# تینوں ایک ساتھ چلتے ہیں
```

## 3.5 Agent SDK میں Hooks

Hooks agent lifecycle میں transformation اور interception پر points کے مخصوص

PostToolUse tool result کو model سے intercept تک پہنچانے سے پہلے model کو PostToolUse

```
# کریں formats normalize سے تاریخ کے MCP tools مثال: مختلف
@hook("PostToolUse")
def normalize_dates(tool_result):
    # Unix timestamp -> ISO 8601
    # "Mar 5, 2025" -> "2025-03-05"
    return normalized_result
```

Outgoing-call interception hook policy والے block کو actions کی خلاف ورزی کرنے والے

```
# کریں refunds block مثال: 500$ سے اوپر
@hook("PreToolUse")
def enforce_refund_limit(tool_call):
    if tool_call.name == "process_refund" and tool_call.args.amount > 500:
        return redirect_to_escalation(tool_call)
```

Hooks اور prompt instructions میں فرق

| Attribute       | Hooks                                                     | Prompt instructions                              |
|-----------------|-----------------------------------------------------------|--------------------------------------------------|
| Guarantee       | Deterministic (100%)                                      | Probabilistic (>90%, 100% نہیں)                  |
| کب استعمال کریں | Critical business rules, financial operations, compliance | General preferences, recommendations, formatting |
| مثال            | Refunds > \$500 block کریں                                | "Try to solve before escalating"                 |

استعمال کریں hooks، انہیں prompts — کے مالی، قانونی، یا حفاظتی نتائج کے failure قاعدے: جب

## 4: Model Context Protocol (MCP)

دستاویزات: [MCP](#) | [Tools](#) | [Resources](#) | [Servers](#)

## 4.1 MCP کیا ہے

MCP (Model Context Protocol) Claude کو external systems سے جوڑتا ہے جو open protocol ایک ہے۔ MCP resource types define کرتا ہے:

1. **Tools** — functions (agent actions) کے لیے (CRUD operations, API calls, command execution)
2. **Resources** — context کے لیے (documentation, database schemas, content catalogs)
3. **Prompts** — predefined prompt templates کے لیے tasks عام

## 4.2 MCP Servers

MCP server ایک process جو MCP protocol implement کرتا ہے اور tools/resources جب فراہم کرنا چاہے۔ آپ MCP server سے connect کر سکتے ہیں:

- وہ خود بخود tools تمام
- ایک ساتھ دستیاب ہوتے ہیں tools کے connected servers تمام
- انہیں کیسے استعمال کرے گا model طے کرتی ہیں کے Tool descriptions

## 4.3 MCP Servers کو Configure کرنا

Project configuration ( `.mcp.json` ) — team usage کے لیے:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    },
    "jira": {
      "command": "npx",
      "args": ["-y", "mcp-server-jira"],
      "env": {
        "JIRA_TOKEN": "${JIRA_TOKEN}"
      }
    }
  }
}
```

ملاحظات:

- میں ہوتا ہے version control میں محفوظ ہوتا ہے اور `.mcp.json` project root
- Environment variables ( `${GITHUB_TOKEN}` ) secrets میں استعمال ہوتے ہیں — tokens خود commit نہیں کیے جاتے



- Database schemas (data structure سمجھنے کے لیے)
- Documentation (API references, internal guides)
- Issue/task summaries

موجود data کی ضرورت نہیں پڑتی کے exploratory tool calls سمجھنے کے لیے agent کا فائدہ Resource فراہم کرتا ہے "map" فوری Resource کیا ہے

## 5 اور Claude Code — Configuration Workflows

دستاویزات: [Claude Code](#) | [Memory / CLAUDE.md](#) | [Skills](#) | [MCP](#) | [Hooks](#) | [Sub-agents](#) | [GitHub Actions](#) | [Headless](#)

### 5.1 CLAUDE.md Hierarchy

hierarchy استعمال کرتا ہے اس کی تین سطحی Claude Code میں جو instruction file(s) و CLAUDE.md ہے:

1. User-level: ~/.claude/CLAUDE.md
  - ہر لاگو user صرف اسی
  - نمونہ ہوتا ہے share کہ ذریعہ VCS
  - working style اور preferences ذاتی
2. Project-level: .claude/CLAUDE.md یا root CLAUDE.md
  - ہر لاگو project contributors تمام
  - ہوتا ہے manage کہ ذریعہ VCS
  - Coding standards, testing standards, architectural decisions
3. Directory-level: subdirectories میں CLAUDE.md
  - کہ ساتھ کام ہو تو لاگو ہوتا ہے files کی directory جب اسی
  - conventions کہ اُس حصہ کہ مخصوص codebase

~/.claude/CLAUDE.md میں ملتی ہیں کیونکہ وہ project instructions کو team member عام غلطی: ایک نئے میں رکھی گئی تھیں (user-level) ~/.claude/CLAUDE.md کے بجائے (project-level)

### 5.2 @path Syntax (File Imports)

جو جاتی configuration modular کر سکتا ہے، جس سے refer کے ذریعہ @path کو files بیرونی CLAUDE.md ہے:

## # Project CLAUDE.md

Coding standards @./standards/coding-style.md میں بیان کیے گئے ہیں  
Test requirements @./standards/testing-requirements.md میں ہیں  
Project overview @README.md اور dependencies @package.json میں ہیں

### @path قواعد:

- (نہیں space کوئی) file path @ فوراً بعد @
- Relative اور absolute paths supported ہیں
- Relative paths اس مطابق file resolve کے میں جس میں import @
- Maximum import nesting depth 5

شامل @ @ standards میں صرف متعلق package کم @ @ اور duplication اس سے

## 5.3 .claude/rules/ Directory

کرنے کے @ @ organize کے حساب سے @ @ topic کو rules کا متبادل @ @، جو monolithic CLAUDE.md @./claude/rules/ لیے استعمال ہوتا ہے:

```
.claude/rules/  
testing.md          -- testing conventions  
api-conventions.md  -- API conventions  
deployment.md       -- deployment rules  
react-patterns.md   -- React patterns
```

### YAML frontmatter کے ساتھ @ @ paths conditional loading:

```
---  
paths: ["src/api/**/*"]  
---
```

استعمال کریں @ explicit error handling کے ساتھ async/await کے لیے API files  
واپس کریں @ endpoint standard response wrapper

```
---  
paths: ["**/*.test.tsx", "**/*.test.ts"]  
---
```

استعمال ہونہ چاہئیں @ describe/it blocks میں Tests  
نہیں @ hardcoding، استعمال کریں Data factories  
استعمال کریں @ test database - نہ کریں Database mock

: یہ کیسے کام کرنا



- کرتا match سہ pattern paths کر جو file edit ایسا Claude Code ہوتا ہے جب load صرف اس وقت Rule ہو
- نہیں ہیں rules load ہوتا ہے — غیر متعلقہ tokens اور context اس سہ
- Glob patterns کرنے دینے ہیں، چاہے conventions apply کے لحاظ سہ file type آپ کو میں codebase files (tests, migrations) پر
- (کے لیے بہت مفید tests) کے ہیں بھی ہوں

کے استعمال کریں directory-level CLAUDE.md بمقابلہ .claude/rules/ والے paths

- .claude/rules/ with paths — جب conventions کئی directories میں بکھری files (tests, migrations) پر لاگو ہوں
- Directory-level CLAUDE.md — جب conventions خاص کسی directory اور کے ہیں اور کے ہیں ضروری نہیں ہوں

## 5.4 Custom Slash Commands اور Skills

skills کو (.claude/commands/) custom commands میں Claude Code version نوٹ: موجود ہے  
 بناتے commands /name formats کر دیا گیا ہے دونوں unify کے ساتھ (.claude/skills/) supported اب بھی format کا ذکر کرتی ہے — و (.claude/commands/) ہیں امتحانی گائیڈ

Slash commands reusable prompt templates ہیں جو /name کے ذریعے invoke کے ہیں:

**.claude/commands/ format (legacy, supported):**

```
.claude/commands/
review.md      -- /review -- standard code review
test-gen.md    -- /test-gen -- test generation
```

**.claude/skills/ format (current):**

```
.claude/skills/
review/SKILL.md -- /review -- frontmatter configuration
test-gen/SKILL.md
```

**Project commands** (.claude/commands/ یا .claude/skills/):

- کرنے والے سب کے لیے دستیاب ہوتا ہے ہیں repo clone میں محفوظ ہوتا ہے ہیں اور VCS
- یقینی بناتے ہیں consistent workflows ٹیم میں

**User commands** (~/.claude/commands/ یا ~/.claude/skills/):

- نہیں ہیں ہوتا ہے share کے ذریعے VCS جو commands ذاتی
- کے لیے workflows انفرادی

## 5.5 Skills — .claude/skills/

Skills advanced commands ہیں جو SKILL.md frontmatter کے ذریعے configure کے ہیں:

```
---
context: fork
allowed-tools: ["Read", "Grep", "Glob"]
argument-hint: "Path to the directory to analyze"
---
```

کا تجزیہ کریں۔ code structure میں directory دی گئی  
report پر ایک architectural patterns اور Dependencies

#### Frontmatter parameters:

| Parameter                                   | Description                                                                                                         |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>context:</code><br><code>fork</code>  | Verbose output main session کو pollute کرتا isolated subagent کو Skill                                              |
| <code>allowed-</code><br><code>tools</code> | نہیں کر skill files delete مثلاً — security) دستیاب ہوں گے tools یہ محدود کرتا کہ کون سے<br>(سکا گی اگر اجازت نہ ہو |
| <code>argument-</code><br><code>hint</code> | مانگنے کا اشارہ argument ہو تو invoke کہ بغیر skill parameters جب                                                   |

#### Skill `CLAUDE.md` کے استعمال کریں:

- Skill — task مخصوص — on-demand invocation (review, analysis, generation) کے لیے
- `CLAUDE.md` — general standards اور conventions ہمیشہ لوڈ ہونے والے

#### Personal skills ( `~/claude/skills/` ):

- متاثر نہ ہوں teammates مختلف ناموں سے بنائیں تاکہ personal variants اپنی

## 5.6 Planning Mode `Direct Execution` کے مقابلہ

#### Planning mode:

- Model نہایت changes کرتا؛ plan اور investigate صرف
- Codebase explore کے لیے Read, Grep, Glob استعمال کرتا
- کرتا approve تیار کرتا جس implementation plan ایک
- safe exploration کے side effects بغیر

#### Planning mode کے استعمال کریں:

- changes (files درجنوں) بڑے
- Multiple plausible approaches (microservices: boundaries کیسے define ہوں؟)
- Architectural decisions (کیا framework کون سا؟ structure؟)
- Unfamiliar codebase (change کی سمجھنا ضروری ہے)
- Library migrations 45+ files کو متاثر کریں

#### Direct execution کے استعمال کریں:

- Single-file fixes ساتھ stack trace واضح
- شامل کرنا validation check ایک
- changes اچھی طرح سمجھ گئی، غیر مبہم

#### Combined approach:

1. Investigation اور design لیے planning mode
2. User plan approve کر
3. Approved plan implement لیے direct execution کرنے

#### Explore subagent — codebase explore لیے specialized subagent:

- Verbose output کو main context سے isolate کرنا
- واپس کرنا summary صرف
- روکنا Multi-phase tasks میں context-window exhaustion

## 5.7 /compact Command

`/compact` context compress لیے built-in command:

- میں جگہ خالی کرنا context window کو summarize کر کے history چھپی
- Long investigation sessions میں verbose tool output سے context بھر جائے استعمال ہوتا ہے جب
- Risk: exact numeric values, dates, اور specific details summarization میں گم ہو سکتے ہیں

## 5.8 /memory Command

`/memory` sessions memory manage لیے built-in command:

- محفوظ کیے جا سکیں context اور notes, preferences, تاکہ `CLAUDE.md` file editing کھولتا ہے،
- Information sessions اور load پر خود بخود startup کے پار برقرار رہتی ہے اور
- Project conventions, user preferences, frequently used commands, اور current work context محفوظ کر کے لیے مفید
- دوبارہ سمجھانے کا متبادل instructions میں وہی session رہے

## 5.9 Claude Code CLI for CI/CD

`-p` (یا `--print`) flag:

```
claude -p "Analyze this pull request for security issues"
```

- Non-interactive mode: prompt process کرنا، اور print پر stdout، exit کرنا،
- User input کا انتظار نہیں کرتا
- CI/CD pipelines میں Claude کا واحد درست طریقہ

CI کے لیے structured output:

```
claude -p "Review this PR" --output-format json --json-schema
'{"type":"object",...}'
```

- `--output-format json` — output کو JSON میں دیتا ہے
- `--json-schema` — output کو schema کے مطابق validate کرتا ہے
- Result کو automatically inline PR comments میں لے کر پوسٹ کر دیتا ہے

**Session context isolation:** Claude session میں جس code generate کیا گیا ہے، اکثر review میں کمی ہوتی ہے، چونکہ review کے وقت challenge ساتھ رکھتا ہے اور اپنی reasoning context (model کی ہوتی ہے) استعمال کرتا ہے۔ independent instance کے لیے Review کے لیے استعمال کریں۔

**Duplicate comments** review results context کے وقت، پچھلے review کے بعد دوبارہ commits سے بچاؤ: unresolved issues report کی ہدایت دیں یا Claude میں شامل کریں اور

## 5.10 Session Management اور `fork_session`

`--resume <session-name>` named session کو resume کرتا ہے:

```
claude --resume investigation-auth-bug
```

- saved context کے ساتھ پچھلی conversation کے جاری رکھتا ہے
- Long investigations کے لیے مفید sessions جو متعدد پر پھیلی ہوئی ہیں ان کے لیے
- Risk: tool results stale بدل چکی ہیں تو files کے بعد session اگر پچھلی

`fork_session` shared context سے independent branch بناتا ہے:

```
Codebase investigation
|
fork_session
/      \
Approach A:  Approach B:
Redux       Context API
```

- کرتا ہے کہ context inherit تک forks branch point دونوں
- کرتا ہے کہ diverge اس کے بعد وہ آزادانہ طور پر
- Approaches compare کے لیے مفید strategies test کرنے یا

**Resume** نئی session کے بجائے نئی

- Tool results stale (files بدل چکی ہیں)
- ہو گیا کہ context degrade کافی وقت گزر چکا ہو اور
- کرنے کے بجائے یہ بہتر ہے کہ "م نہ جو پایا اس کا مختصر خلاصہ" یا "restart ..."

# باب 6: Prompt Engineering — Advanced Techniques

دستاویزات: [Prompt Engineering](#) | [Anthropic Cookbook](#)

## 6.1 Few-shot Prompting

Few-shot prompting میں input/output prompt 2–4 دکھانے کے لیے expected behavior کے طریقے جس میں شامل کیے جاتے ہیں examples

**Few-shot text descriptions** زیادہ مؤثر کیوں

- "وہ" کو کئی طریقوں سے سمجھا جا سکتا ہے precise جیسے "زیادہ" instruction میں
- Example decision logic اور expected format غیر میں طور پر
- Model pattern (صرف examples) کرتا ہے generalize پر cases کو نئے

**Few-shot examples** کی اقسام اور استعمال

1. Ambiguous scenarios کے لیے examples:

```
Request: "My order is broken"
Action: Call get_customer -> lookup_order -> check status.
Rationale: "broken" damaged item کو آپ ہو سکتا ہے؛ order details میں درکار
```

```
Request: "Get me a manager"
Action: escalate_to_human call کریں فوراً
Rationale: Customer مانگ رہا ہے خود حل کرنے کی کوشش نہ کریں human واضح طور پر
```

2. Output formatting کے لیے examples:

```
Finding example:
{
  "location": "src/auth/login.ts:42",
  "issue": "SQL injection in the username parameter",
  "severity": "critical",
  "suggested_fix": "Use a parameterized query"
}
```

3. Acceptable problematic code کے لیے examples:

```
// Acceptable (flag نہ کریں):
const items = data.filter(x => x.active);

// Problem (flag کریں):
const items = data.filter(x => x.active == true); // Use strict equality ===
```

4. Different document formats سے extraction کے examples:

Document with inline citations:

"As shown in the study (Smith, 2023), the rate is 42%."

-> {"value": "42%", "source": "Smith, 2023", "type": "inline\_citation"}

Document with bibliography references:

"The rate is 42%. [1]"

-> {"value": "42%", "source": "reference\_1", "type": "bibliography"}

## 5. Informal measurements کے لیے examples:

Text: "about two handfuls of rice"

-> {"amount": "~100g", "original\_text": "two handfuls", "precision": "approximate"}

Text: "a pinch of salt"

-> {"amount": "~1g", "original\_text": "a pinch", "precision": "approximate"}

Few-shot purely rule-based instructions میں مؤثر اور non-standard measurement units اور informal خاص طور پر Few-shot based instructions کے لیے بہت متنوع ہوتی ہیں۔

**Prompts استعمال strict JSON schemas کے لیے structured output جب format normalization rules میں:** شامل کریں normalization rules میں prompt کریں:

Normalization:

- Dates: ISO 8601 (YYYY-MM-DD); "yesterday" -> absolute date compute کریں ہمیشہ
- Currency: numeric amount + currency code; "five bucks" -> {"amount": 5, "currency": "USD"}
- Percentages: decimal fraction; "half" -> 0.5

وہ values inconsistent درست ہو مگر JSON syntactically کم ہوتے ہیں جہاں semantic errors اس سے

## 6.2 Explicit Criteria بمقابلہ Vague Instructions

**Bad (vague):**

Code comments کی accuracy چیک کریں

Conservative high-confidence findings report کریں - صرف

**Good (explicit criteria):**

Comment کو problematic میں اس صورت میں flag جب:

1. Comment کا actual code behavior سے CONTRADICT کرتا ہو
2. Comment کا حوالہ دہ variable یا function کسی غیر موجود
3. TODO/FIXME comment ایسے bug جو کسی fix میں پہلے ہی code کی طرف اشارہ کرے جو

Flag نہ کریں:

- کہ لحاظ سے پرانہ ہوں style جو صرف comments ایسے
- wording inaccuracies معمولی
- Missing comments (الگ category یہ الگ)

Severity criteria examples کے ساتھ define کریں:

CRITICAL: User کے لیے runtime failure

Example: Payment process میں NullPointerException

HIGH: Security vulnerability

Example: SQL injection, XSS, missing authorization checks

MEDIUM: Logic bug کا بغیر فوری impact

Example: Wrong sorting, off-by-one error

LOW: Code quality

Example: Duplication, suboptimal algorithm کا لیے data چھوٹا

## 6.3 Prompt Chaining

Prompt chaining میں توڑنا sequence کی focused steps کو ایک complex task

Step 1: auth.ts analyze کریں (صرف local issues)

-> Output: auth.ts میں issues کی list

Step 2: database.ts analyze کریں (صرف local issues)

-> Output: database.ts میں issues کی list

Step 3: Integration pass (cross-file dependencies)

-> Output: module boundaries پر issues

یہ کیوں اہم:

- **Attention dilution** — جب files دیکھتا تو کچھ files ایک ساتھ ہوت سی model سے بچاتا — جب bugs miss files دیکھتا تو کچھ files ایک ساتھ ہوت سی model سے بچاتا
- **Shallow commentary** — shallow commentary کر سکتا اور باقی پر
- **Consistent analysis quality** — consistent analysis quality کا لیے file
- **Cross-file interactions** — الگ analysis کا لیے الگ

Prompt chaining کا استعمال کریں dynamic decomposition بمقابلہ

- **Prompt chaining** — predictable, repeatable tasks (code review, file migrations)
- **Dynamic decomposition** — open-ended investigations ان subtasks execution کے دوران واضح ہوتے ہیں

## 6.4 “Interview” Pattern

Solution implement کا پڑھنا Claude clarifying questions کرنے کا پڑھنا

Claude: "API کا لیے caching implement کرنے سے پہلے چند سوالات

1. event-based یا TTL آپ پسند کرتے ہیں؟ cache invalidation strategy میں سے کون سی
2. stale data ہو تو کیا cache unavailable اگر قابل قبول ہے؟
3. Caching per-user یا global؟ مونی چاہیے
4. Cache کا لیے expected data volume کیا ہے؟

یہ کب مفید:

- Unfamiliar domain (fintech, healthcare, legal systems)
- Implications and tasks (cache strategies, failure modes)
- Viable approaches context جہاں بہترین انتخاب

## 6.5 Validation اور Retry-with-Feedback

جب extracted data validation fail کرے:

Step 1: Document سے data extract کریں  
 Step 2: Validate کریں (Pydantic, JSON Schema, business rules)  
 Step 3: retry کریں کہ ساتھ context – ہو error اگر:  
 - Original document  
 - Previous (incorrect) extraction  
 - Specific error: "Field 'total' = 150, but sum(line\_items) = 145. Re-check values."

**Retry مؤثر ہوگا:**

- Format errors (date غلط format میں)
- Structural errors (field جگہ غلط)
- Arithmetic inconsistencies (model دوبارہ check کر سکتا ہے)

**Retry گا مدد نہیں کرے:**

- Information source document نہیں ہے
- Required context external ہے (data کسی اور document میں کیا گیا)

**Pydantic validation tool:** Pydantic Python library جو schema-based data validation کے لیے استعمال ہوتی ہے امتحان کے لیے اس کے نکات یہ ہیں:

- **Structural validation:** types, requiredness, enum constraints JSON کے بعد code میں چیک کیے جاتے ہیں
  - **Semantic validation:** custom validators business logic enforce کرتے ہیں (sum of items equals total! start\_date < end\_date)
  - **Validate-retry loops:** Pydantic validation failure پر error message بنا کر Claude کو error context کے ساتھ prompt دوبارہ کریں
  - **JSON Schema generation:** Pydantic models سے JSON Schema generate کر سکتے ہیں `tool_use` کے لیے
- فراہم کرتا ہے single source of truth، جو ایک

## 6.6 Self-correction

Internal contradictions detect کرنے کا pattern:



```
{
  "stated_total": "$150.00",
  "calculated_total": "$145.00",
  "conflict_detected": true,
  "line_items": [
    {"name": "Widget A", "price": 75.00},
    {"name": "Widget B", "price": 70.00}
  ]
}
```

آپ `conflict_detected` دونوں نکالتا ہے — اگر وہ مختلف ہوں تو Model stated value اور computed value کو discrepancy handle کرنے دیتا ہے

## 7 Message Batches API: باب 7

دستاویزات: [Message Batches](#)

### 7.1 Overview

کرنے دیتا ہے batches submit کے requests کے لیے asynchronous processing کو Message Batches API

| Attribute               | Value                                                                    |
|-------------------------|--------------------------------------------------------------------------|
| Savings                 | 50% کے مقابلے میں 50 synchronous calls                                   |
| Processing window       | 24 hours (latency SLA guarantee) زیادہ سے زیادہ                          |
| Multi-turn tool calling | Supported نہیں (یک request = ایک response)                               |
| Correlation             | request اور response کے لیے <code>custom_id</code> field کو جوڑنے کے لیے |

### 7.2 Batch API کے مقابلے Synchronous API میں استعمال کریں

| Task                          | API         | Why                                                 |
|-------------------------------|-------------|-----------------------------------------------------|
| Pre-merge PR check            | Synchronous | Developer 24 hours انتظار کر رہا ہے؛ قابل قبول نہیں |
| Overnight tech-debt report    | Batch       | savings % صبح تک چاہیے؛ Result 50                   |
| Weekly security audit         | Batch       | savings % فوری نہیں؛ 50                             |
| Interactive code review       | Synchronous | درکار کے response فوری                              |
| 10,000 documents process کرنا | Batch       | Bulk processing؛ savings میں                        |

## 7.3 custom\_id کا استعمال

```
{
  "custom_id": "doc-invoice-2024-001",
  "params": {
    "model": "claude-sonnet-4-6",
    "max_tokens": 1024,
    "messages": [{"role": "user", "content": "Extract data from: ..."}]
  }
}
```

custom\_id آپ کو یہ کرنے دیتا ہے:

- Result کو original document link سے
- Failure submit دوبارہ کی صورت میں صرف
- Successful documents process کو دوبارہ

## 7.4 Batch کو Failures میں Handle کرنا

1. 100 documents کا batch submit کریں
2. 95 succeed 5 fail (context limit exceeded) ہوں
3. Failures کو custom\_id سے identify کریں
4. Strategy (میں تقسیم کریں chunks کو long documents مثلاً) بدلیں
5. 5 failed documents submit کو دوبارہ صرف اُن

## 7.5 SLA Planning

تک لگ سکتے ہیں 24 hours کو Batch API چاہے اور result میں hours اگر آپ کو 30

- Submission window: 30 - 24 = 6 hours
- Batches کو deadline 24 hours سے کم از کم
- Frequent submissions 4 hour windows میں تقسیم کریں

# 8: Task Decomposition Strategies

## 8.1 Fixed Pipelines (Prompt Chaining)

ہوتا ہے defined step سے:

Document -> Metadata extraction -> Data extraction -> Validation -> Enrichment -> Final output

کب استعمال کریں:

- Task structure predictable (reviews ایک template follow کریں)
- پلاٹ سڈ معلوم ہوں steps تمام
- چا پیڈ reproducibility اور stability آپ کو

## 8.2 Dynamic Adaptive Decomposition

Subtasks intermediate results پر بنیاد کی generate ہوتے ہیں:

1. "Legacy codebase کے لیے tests add کریں"
2. -> structure map (Glob, Grep) کریں
3. -> 3 modules بغیر partial coverage کے، 2 tests
4. -> Prioritize: payments module شروع کریں (high risk)
5. -> دریافت ہوئی dependency پر external API: کام کے دوران
6. -> Adapt: tests کے لیے mock add کریں external API لکھنے سے پہلے

کب استعمال کریں:

- Open-ended investigative tasks
- شروع میں معلوم نہ ہو scope جب مکمل
- پر منحصر ہوں results کے step پچھلے step جب ر

## 8.3 Multi-pass Code Review

10+ files والی pull requests کے لیے:

```
Pass 1 (per-file): auth.ts analyze کریں -> local issues کی list
Pass 1 (per-file): database.ts analyze کریں -> local issues کی list
Pass 1 (per-file): routes.ts analyze کریں -> local issues کی list
...
Pass 2 (integration): files درمیان کے relationships analyze کریں
-> Cross-file issues: inconsistent types, circular dependencies
```

14 files کی pass پر ایک کی خراب:

- Attention dilution: shallow کچھ کی، deep analysis کی files کچھ
- Inconsistent comments: approve دوسری میں، pattern flag میں ایک file
- Missed bugs: cognitive overload واضح ہوں errors کی وجہ سے

# 9 Escalation اور Human-in-the-Loop: باب 9

## 9.1 Escalate کی طرف Human کب

Escalation triggers (rules واضح):

| Situation | Action |
|-----------|--------|
|-----------|--------|

|                                                 |                                                                     |
|-------------------------------------------------|---------------------------------------------------------------------|
| Customer "get me a manager" واضح طور پر کہتا ہے | کریں؛ حل کرنے کی کوشش نہ کریں escalate فوراً                        |
| Policy request کو cover نہ کرتی ہو              | Escalate policy جب competitor price matching مثلاً کریں (ہو)        |
| Agent پیش رفت نہ کر سکے                         | کریں escalate کے بعد attempts مناسب تعداد میں                       |
| Threshold financial operation سے اوپر           | Escalate کے ذریعے prompt enforce، کے ذریعے hook ترجیحاً کریں (نہیں) |
| Customer multiple matches تلاش کرتے وقت         | مانگیں؛ انداز نہ لگائیں identifiers مزید                            |

نہیں ہیں reliable trigger جو:

| Unreliable method                  | Why it fails                                            |
|------------------------------------|---------------------------------------------------------|
| Sentiment analysis                 | نہیں کرتا correlate سے Customer کا mood case complexity |
| Model self-rated confidence (1–10) | کمزور calibration غلط ہو سکتا ہے؛ Model confidently     |
| Automatic classifier               | موجود نہ ہو training data؛ ممکن Overengineering         |

## 9.2 Escalation Patterns

Immediate escalation:

Customer: "I want to speak to a manager"  
 Agent: [کرتا escalate\_to\_human call فوراً]  
 NOT: "I can help with your issue, let me..."

Resolve escalation: کی کوشش کے بعد

Customer: "My refrigerator broke two days after purchase"  
 Agent: [کرتا warranty replacement offer، چیک کرتا]  
 escalate -> مطمئن نہ ہو customer اگر

Nuanced escalation (acknowledge → resolve → reiteration پر escalate):

Customer: "This is outrageous, I'm very unhappy with the quality!"  
 Agent: [frustration acknowledge] "سمجھتا ہوں frustration میں آپ کی"  
 [resolution offer] "replacement یا refund ہو سکتا ہے"  
 Customer: "No, I want to talk to someone!"  
 Agent: [customer insist دوبارہ -> immediate escalation]

کریں escalate دیں، اور صرف اسی وقت concrete solution کریں، پھر emotion acknowledge بنیادی اصول: پہلے  
 manager (نہیں) کریں escalate کے پہلے اطلاع پر dissatisfaction کی خواہش دہرائیں customer human جب  
 (مانگندہ کے برابر نہیں ہیں)

Policy gap کے لیے escalation:

Agent: [escalate – policy competitor price matching کو cover کرتی نہیں]

- High-confidence extractions □□ sample audit کریں
- Aggregate 97% accuracy □□ errors □□ document type 40 لیجیے
- Accuracy □□ analyze کریں □□ field اور document type □□، overall کو صرف

# Multi-agent Systems میں Error Handling: باب 10

## 10.1 Error Categories

| Category   | Examples                                     | Retryable           | Agent action                           |
|------------|----------------------------------------------|---------------------|----------------------------------------|
| Transient  | Timeout, 503, network failure                | Yes                 | Exponential backoff ساتھ retry کریں    |
| Validation | Invalid input format, missing required field | No (input fix کریں) | Request modify اور retry کریں          |
| Business   | Policy violation, threshold exceeded         | No                  | پیش alternative کو سمجھائیں؛ User کریں |
| Permission | Access denied                                | No                  | Escalate کریں                          |

## 10.2 Error-handling Anti-patterns

| Anti-pattern                                | Problem                                                     | Correct approach                                           |
|---------------------------------------------|-------------------------------------------------------------|------------------------------------------------------------|
| Generic status “search unavailable”         | Coordinator decide نہیں کر سکتا کہ کیسے recover کرنا ہے     | Error type, query, partial results, alternatives واپس کریں |
| Silent suppression (empty result = success) | Coordinator match سمجھتا ہے کہ ہوا تھا failure ملا، حالانکہ | “no results” اور “search failure” میں فرق کریں             |
| Workflow پر پورا failure ایک کرنا abort     | ضائع ہو جاتا ہے partial results تمام                        | Partial results ساتھ جاری رکھیں؛ gaps annotate کریں        |
| Subagent میں infinite retries               | Latency اور resources کا ضیاع                               | Local recovery (1–2 retries)، پھر coordinator تک propagate |

## 10.3 Structured Subagent Error

```
{
  "status": "partial_failure",
  "failure_type": "timeout",
  "attempted_query": "AI impact on music industry 2024",
  "partial_results": [
    {"title": "AI Music Generation Report", "url": "...", "relevance": 0.8}
  ],
  "alternative_approaches": [
    "Try a narrower query: 'AI music composition tools'",
    "Use an alternative data source"
  ],
  "coverage_impact": "Not covered: AI impact on music production"
}
```

کو فیصلہ کرنا کہ آیا ضروری معلومات دیتا ہے coordinator

- Modified query ساتھ retry کریں؟
- Partial results استعمال کریں؟
- subagent کو delegate کسی اور کریں؟
- gap annotate کریں؟ بغیر جاری رکھیں اور section اس

## 10.4 Final Synthesis میں Coverage Annotations

**## Report: AI Impact on Creative Industries**

**### Visual Art (FULL COVERAGE)**

[research results]

**### Music (PARTIAL COVERAGE – search agent timeout)**

[partial results]

⚠️ Note: coverage search agent timeout سے محدود ہے اس section کی

**### Literature (FULL COVERAGE)**

[research results]

# Context میں Production Systems: باب 11

## Management

### 11.1 Facts کو Extract میں Separate Block

Conversations history کو (جو summarize کے دوران summarize ہو جاتی ہے) key facts کو extract میں structured block میں:

```
=== CASE FACTS (fact کو جب بھی نیا) ===
Customer ID: CUST-12345
Order ID: ORD-67890
Order Date: 2025-01-15
Order Amount: $89.99
Issue: Damaged item on delivery
Customer Request: Full refund
Status: Pending manager approval
===
```

کیونکہ یہ summarized ہے، history میں شامل کریں، prompt میں block

### 11.2 Tool Results کو Trimming کرنا

اگر `lookup_order` 40+ fields مگر current task میں صرف 5 درکار ہیں واپس کرنا

```
# PostToolUse hook: relevant fields رکھیں
@hook("PostToolUse", tool="lookup_order")
def trim_order_fields(result):
    return {
        "order_id": result["order_id"],
        "status": result["status"],
        "total": result["total"],
        "items": result["items"],
        "return_eligible": result["return_eligible"]
    }
```

کم ہوتی ہے noise بچتا ہے اور context اس سے

### 11.3 Position-aware Input

Lost-in-the-middle effect کو critical information میں رکھتے ہوئے



[KEY FINDINGS – اوپر]

3 critical vulnerabilities ملیں...

[DETAILED RESULTS – درمیان]

=== File auth.ts ===

...

=== File database.ts ===

...

[ACTION ITEMS – آخر میں]

Priority: merge auth.ts vulnerabilities fix کریں.

## 11.4 Scratchpad Files

Long investigations میں agent key findings scratchpad file میں لکھ سکتا ہے:

```
# investigation-scratchpad.md
## Key findings
- PaymentProcessor in src/payments/processor.ts inherits from BaseProcessor
- refund() تین places میں call ہوئی ہے: OrderController, AdminPanel, CronJob
- External PaymentGateway API limit: 100 req/min
- Migration #47 میں refund_reason (NOT NULL) add کیا گیا - 01-12-2024
```

scratchpad دوبارہ چلانے کے بجائے agent discovery (میں session یا نئی) کو جائے context degrade جب consult کر سکتا ہے

## 11.5 Context کے لیے Subagents کو Delegate کرنا

Main agent: "payments module کی dependencies investigate کریں"

-> Subagent (Explore): 15 files read کرتا ہے، imports trace کرتا ہے

-> Returns: "Payments depends on AuthService, OrderModel, and the external PaymentGateway API"

Main agent: context 15 میں files کے بجائے ایک line کے رکھتا ہے

**Separate context layer:** Multi-agent systems میں subagent context budget — میں کام کرتا ہے — Coordinator ملتی ہے information کے لیے ضروری task اسے صرف اپنے کے طور پر separate context layer ایک global state store، اور context allocate کرتا ہے، subagent outputs aggregate کرتا ہے، اور "context leakage" اس سے بھر دیتا ہے جو information کو ایسی agent window میں ہوتا، جہاں ایک "context leakage" اس سے دوسروں کے لیے غیر متعلق ہے

**Subagents کے لیے constrained context budgets:**

- Minimal context: specific task + ضروری data
- Subagent کو raw data dumps کے بجائے structured results کی ہدایت دینے کے لیے
- اور کم distractions کا مطلب کم tools کریں — کم limit کو toolset کے subagent سے allowedTools context cost

## 11.6 Structured State Persistence (crash recovery لی کرنا)

کرنا export پر location ایک معلوم state اپنی agent ر:

```
// agent-state/web-search-agent.json
{
  "status": "completed",
  "queries_executed": ["AI music 2024", "AI music composition"],
  "results_count": 12,
  "key_findings": [...],
  "coverage": ["music composition", "music production"],
  "gaps": ["music distribution", "music licensing"]
}
```

Coordinator resume پر ایک manifest load کرنا:

```
// agent-state/manifest.json
{
  "web-search": "completed",
  "doc-analysis": "in_progress",
  "synthesis": "not_started"
}
```

## 12 کو محفوظ رکھنا Provenance: باب 12

### 12.1 Attribution Loss Problem

گم ہو سکتا ہے link "claim → source" کی جاتی ہیں، تو results summarize سے sources جب متعدد

Bad: "The AI music market is estimated at \$3.2B." (No source, no year.)

Good:

```
{
  "claim": "The AI music market is estimated at $3.2B.",
  "source_url": "https://example.com/report",
  "source_name": "Global AI Music Report 2024",
  "publication_date": "2024-06-15",
  "confidence": 0.9
}
```

### 12.2 Conflicting Data کو Handle کرنا

دیں values مختلف sources جب دو

```
{
  "claim": "Share of AI-generated music on streaming platforms",
  "values": [
    {
      "value": "12%",
      "source": "Spotify Annual Report 2024",
      "date": "2024-03",
      "methodology": "Automated classification"
    },
    {
      "value": "8%",
      "source": "Music Industry Association Survey",
      "date": "2024-07",
      "methodology": "Survey of 500 labels"
    }
  ],
  "conflict_detected": true,
  "possible_explanation": "Methodology اور time period فرق کا"
}
```

کو coordinator کے ساتھ محفوظ رکھیں اور attribution کو اندھا دھند منتخب نہ کریں۔ دونوں کو value کسی ایک فیصلہ کرنے دیں۔

## 12.3 Correct Interpretation کے لیے Dates شامل کریں

سمجھ لیا جا سکتا ہے کہ contradiction کو temporal differences کے بغیر Dates

Bad: "Source A says 10%, source B says 15%. Contradiction."

Good: "Source A (2023) says 10%, source B (2024) says 15%. Likely 5+ سال میں 1% growth."

## 12.4 Content Type کے مطابق Render کریں

میں نہ ٹھونسیں format کے چیز کو ایک کی

- Financial data -> tables
- News اور analysis -> prose
- Technical findings -> structured lists
- Time series -> chronological ordering

## باب 13: Claude Code کے Built-in Tools

### 13.1 Tool Selection Reference

| Task                              | Tool         | Example                                                          |
|-----------------------------------|--------------|------------------------------------------------------------------|
| تلاش کرنا files کے نام/pattern    | <b>Glob</b>  | <code>**/*.test.tsx</code> , <code>src/components/**/*.ts</code> |
| File contents کے اندر search کرنا | <b>Grep</b>  | Function name, error message, import                             |
| File مکمل پڑھنا                   | <b>Read</b>  | Analysis کے لیے file load کرنا                                   |
| بنانا file نیا                    | <b>Write</b> | Scratch کے file create کرنا                                      |
| precise edit میں file موجود       | <b>Edit</b>  | Unique text match کے ذریعے specific snippet replace کرنا         |
| Shell command چلانا               | <b>Bash</b>  | git, npm, tests, build                                           |

### 13.2 Incremental Investigation Strategy

نہ: پڑھیں کے سمجھ بوجھ کو رفتہ رفتہ بنائیں files ایک ساتھ تمام

1. Grep: entry points تلاش کریں (function definition, export)
2. Read: پڑھیں files ملنے والی
3. Grep: usages تلاش کریں (import, calls)
4. Read: consumer files پڑھیں
5. کریں repeat نہ مل جائے picture جب تک مکمل

### 13.3 Fallback: Edit کے بجائے Read + Write

و: جائے fail کی وجہ سے Edit non-unique text match جب

1. Read — لوڈ کریں file content مکمل
2. Content کو programmatically modify کریں
3. Write — لکھ دیں updated version

## امتحانی ڈومین نوٹس: II حصہ

# 1 Agent Architecture اور Orchestration ڈومین 1 (27%)

## 1.1 Autonomous Task Execution لیجیٹیم آگنٹک لوپس ڈیزائن کرنا

### Key knowledge:

- Agent loop lifecycle: Claude request بھیجیں، `stop_reason` ("tool\_use" بمقابلہ "end\_turn") چیک واپس کریں results کی لیجیٹیم iteration کریں، اگلی tools execute کریں
- Tool results conversation history میں append کیے تاکہ model آگلا action کر سکے
- Model-driven decision making (Claude آگلا tool چنتا) بمقابلہ hard-coded decision trees

### Key skills:

- Flow control: جب `stop_reason = "tool_use"` تو loop جاری رکھیں اور `"end_turn"` پر رک جائیں
- Iterations درمیان context کو tool results میں append کرنا
- Anti-patterns سے بچنا: completion کی لیجیٹیم assistant text parse کرنا، primary stopping mechanism کے استعمال پر arbitrary iteration limits

## 1.2 Multi-agent Systems (Coordinator–Subagent) Orchestrate کرنا

### Key knowledge:

- Hub-and-spoke architecture: coordinator تمام inter-agent communication، error handling، اور routing کا مالک ہوتا ہے
- Subagents isolated context میں کام کرتے ہیں — خود بخود coordinator کی history inherit نہیں کرتے
- Coordinator responsibilities: task decomposition، delegation، result aggregation، dynamic selection of subagents
- Coordinator کی overly narrow decomposition کا خطرہ

### Key skills:

- Research coverage کم ہے duplication کی درمیان تقسیم کریں تاکہ subagents کو
- Iterative refinement loops implement کریں (coordinator synthesis evaluate اور tasks re-route کریں)
- Observability کے ذریعے communication coordinator کی لیجیٹیم تمام

## کریا

### Key knowledge:

- Task tool subagents spawn: □□ کرتا coordinator □□ allowedTools میں "Task" □□ شامل ہونا چاہیے
- Subagent context prompt میں explicitly؛ □□ ہونا چاہیے؛ subagents parent context inherit □□ کرتے ہیں
- AgentDefinition configuration: descriptions, system prompts, tool constraints
- Alternatives explore □□ لپ □□ کرنا؛ fork\_session □□ ذریعہ session management

### Key skills:

- Previous agents کا full outputs subagent prompt میں شامل کریں
- Context pass کو structured formats میں لے کر metadata اور data کو الگ الگ کرتے وقت استعمال کریں
- Coordinator turn میں متعدد Task calls کے ذریعے parallel subagents spawn کریں
- Coordinator prompts goals اور quality criteria کے طور پر ہیں، step-by-step instructions کے طور پر نہیں

## Workflows Implement کرنا

### Key knowledge:

- Workflow ordering اور **prompt guidance** (hooks, preconditions) کے **programmatic enforcement** کے لیے درمیان فرق
- **identity verification** (مثلاً مالی operations پر)، **prompts** کے **deterministic guarantees** درکار ہوں گے۔  
کافی نہیں ہوتے
- **structured handoff protocols** (customer ID, reason, recommended action) کے دوران **Escalation**

### Key skills:

- مکمل نہ ہوں steps کریں جب تک پبلو block کو downstream calls جو programmatic preconditions ایسا (کریں block `process_refund` واپس کرنے تک ID verified کے `get_customer` مثلاً)
- Multi-aspect customer requests الگ items کو کریں
- Human کو escalate وقت structured summaries produce کریں

## 1.5 Tool Calls Intercept اور Agent کرنا کہ لے Data Normalize کرنے کا لے

### Key knowledge:

- Hook patterns (مثلاً `PostToolUse`) tool results کو model consume کرنا پڑے گا۔
- Hooks outgoing calls intercept کر کے compliance rules enforce کرنا پڑے گا۔ (مثلاً threshold اوپر refunds block کرنا)
- Hooks **deterministic guarantees** دیتی ہیں، جبکہ **probabilistic compliance** prompt instructions دیتے ہیں۔

## Key skills:

- `PostToolUse` hooks سے data formats normalize کریں (Unix timestamps, ISO 8601, numeric status codes)
- Policy-violating actions block کرنے کے لیے interception hooks اور escalation استعمال کریں اور redirect کریں
- منتخب کریں hooks کے بجائے prompts چاہیے۔ تو تو guaranteed compliance کو business rules جب

## 1.6 Complex Workflows کے لیے Task Decomposition Strategies

### Key knowledge:

- **Fixed pipelines** (prompt chaining) کے مقابلے میں **dynamic adaptive decomposition** کی بنیاد پر intermediate results
- Prompt chaining: sequential steps (پھر integration pass، analyze کو الگ الگ file)
- Adaptive investigation plans جو discovered information کی بنیاد پر subtasks generate کرتے ہیں

### Key skills:

- Predictable multi-aspect reviews کے لیے prompt chaining؛ open-ended investigations کے لیے dynamic decomposition
- Large code reviews کو per-file analysis + separate cross-file integration pass میں تقسیم کریں
- Open-ended tasks کو degrade ہونے سے پہلے prioritized plan کریں، پھر structure map کریں: پھر

## 1.7 Session State, Resuming, اور Forking

### Key knowledge:

- Named sessions continue کرنے کے لیے `--resume <session-name>`
- Shared context سے independent investigation branches بنانے کے لیے `fork_session`
- Resume کی اہمیت file changes کو agent کرتے وقت
- Structured summary کے ساتھ session، stale results کے ساتھ resuming زیادہ reliable سے زیادہ

### Key skills:

- Named investigation sessions continue کرنے کے لیے `--resume` استعمال کریں
- Approaches parallel compare کرنے کے لیے `fork_session` استعمال کریں
- Resuming (context ابھی current ہے) اور نئی session شروع کرنے (results stale ہیں) درمیان انتخاب کریں

## 5.10 fork\_session اور Session Management

`--resume <session-name>` named session کو resume کرتا ہے:

```
claude --resume investigation-auth-bug
```

- جاری رکھتا conversation کے ساتھ پچھلی saved context
- Long investigations sessions جو متعدد پر پھیلی ہوں ان کے لیے مفید
- Risk: tool results stale بدل چکی ہوں تو files کے بعد session اگر پچھلی

**fork\_session** shared context سے independent branch بنانا:

```
Codebase investigation
|
fork_session
/      \
Approach A:  Approach B:
Redux       Context API
```

- کرتے ہیں context inherit تک forks branch point دونوں
- کرتے ہیں diverge اس کے بعد وہ آزادانہ طور پر
- کرنے کے لیے مفید strategies test کرنے یا Approaches compare

**Resume** نئی session کے بجائے نئی:

- Tool results stale ہوں (files بدل چکی ہوں)
- ہو گیا context degrade کافی وقت گزر چکا ہو اور
- کرنے کے بجائے یہ بہتر ہے کہ "م نہ جو پایا اس کا مختصر خلاصہ یہ ہے" resume کے ساتھ tool data پرانے کیا جائے restart "... سے"

## 2 Tool Design اور MCP Integration (18%)

### 2.1 Clear Descriptions کے ساتھ Tool Interfaces Design کرنا

#### Key knowledge:

- Tool descriptions minimal descriptions کرنا؛ LLM tools select میں جن سے mechanism و بنیادی unreliable selection دیتی
- Input formats, example queries, edge cases, اور applicability boundaries کی اہمیت
- Ambiguous یا overlapping descriptions misrouting کا سبب بنتی ہیں
- System prompt wording tools کے associations کے ساتھ غیر ارادی بنا سکتی ہیں

#### Key skills:

- کریں distinguish سے واضح طور پر similar alternatives کو tool لکھیں جو descriptions ایسی
- Functional overlap کے لیے (مثلاً `analyze_content` -> `extract_web_results`) کریں tools rename ختم کرنے کے لیے
- واضح ہوں input/output contracts میں تقسیم کریں جن کے specialized tools کو general-purpose tools



## 2.2 MCP Tools کے لیے Structured Error Responses Implement کرنا

### Key knowledge:

- MCP tool responses میں `isError` flag
- Transient errors** (timeouts), **validation errors** (bad input), **business errors** (policy violations), اور **access/permission errors** کے درمیان فرق
- Generic errors ("Operation failed") میں recovery decisions روک دیتے ہیں
- Retryable اور non-retryable errors میں فرق

### Key skills:

- `errorCategory` (transient/validation/permission), `isRetryable` جیسی human-readable message اور `retryable: false` کے ساتھ user-facing explanations کے لیے واضح structured metadata واپس کریں
- Business-rule violations کے لیے واضح `retryable: false` استعمال کریں
- Transient failures کے لیے subagents میں local recovery کے لیے صرف وہ errors propagate کریں جو خود حل نہ کر سکیں
- Access failures (retry decision) اور valid empty results (no matches) میں فرق کریں

## 2.3 Agents میں Tools Allocation اور `tool_choice` Configure کرنا

### Key knowledge:

- کم کرتی tool selection reliability (مثلاً 18 کے بجائے 4-5) tools کے زیادہ
- Specialization کے لیے agents رکھنے والے tools سے باہر
- Scoped tool access: role-relevant tools + cross-role utilities محدود
- `tool_choice: "auto"`, `"any"` اور forced tool selection ( `{"type": "tool", "name": "..."}`  )

### Key skills:

- تک محدود رکھیں relevant tools صرف subagent کا toolset
- General tools کو constrained alternatives سے بدلیں (مثلاً `fetch_url` -> `load_document` )
- `tool_choice: "any"` کے بجائے text answer کے لیے استعمال کریں تاکہ
- Specific tool force کے لیے execution order assured کریں تاکہ

## 2.4 MCP Servers اور Claude Code میں Agent Workflows کو Integrate کرنا

### Key knowledge:

- MCP server scope: user کے لیے experiments کے مقابلے میں project ( `.mcp.json` ) کے لیے ( `~/claude.json` )
- Secrets management کے لیے `.mcp.json` میں environment variable substitution (مثلاً `${GITHUB_TOKEN}` )

- available ہوتا ہے اور جاننے والے میں اور ایک ساتھ discover پر tools connection کے connected MCP servers تمام ہیں
- MCP resources بطور "content catalogs" (task summaries, database schemas) exploratory tool calls کم کرتے ہیں

### Key skills:

- Shared MCP servers کو project میں `.mcp.json` env-var-based tokens ساتھ configure کے
- Personal/experimental servers کو `~/claude.json` میں رکھیں
- Standard integrations کے لیے community MCP servers کو custom servers پر ترجیح دیں

## 2.5 Built-in Tools (Read, Write, Edit, Bash, Grep, Glob) منتخب اور کرنا Apply

### Key knowledge:

- **Grep**: file contents کے اندر search (function names, error messages, imports)
- **Glob**: name/extension patterns کے تلاش کرنا files کے
- **Read/Write**: full-file operations؛ **Edit**: unique text matches کے ذریعے precise changes
- **Read + Write** پر واپس جائیں تو fail کی وجہ سے کے Edit non-unique matches کے

### Key skills:

- Content search کے لیے Glob اور pattern-based discovery کے لیے Grep
- **Read** کے لیے flows trace پھر Grep کے لیے entry points: سمجھ بوجھ رفتہ رفتہ بنائیں
- Wrapper modules کے ذریعے function usage trace کریں

## 3 اور Claude Code Configuration: ڈومین Workflows (20%)

### 3.1 Hierarchy, Scope, اور Modular Organization کے ساتھ CLAUDE.md Configure کرنا

#### Key knowledge:

- CLAUDE.md hierarchy: user ( `~/claude/CLAUDE.md` ), project ( `.claude/CLAUDE.md` یا root `CLAUDE.md` ), اور directory-level (subdirectories میں CLAUDE.md)
- User-level settings اور user صرف ایک ہیں؛ VCS کے ذریعے share کے لیے لاگو ہوتے ہیں اور
- External files refer کے لیے `@path` syntax (مثلاً `@./standards/coding-style.md`) تاکہ CLAUDE.md modular بن سکے
- Monolithic CLAUDE.md بجائے `.claude/rules/` directory کے لیے topic-focused rule files کے

### Key skills:

- user-level اس لیے میں ملتیں کیونکہ وہ instructions کو team member (نہ) کریں Hierarchy issues diagnose (میں نہ) میں تھیں، project-level
- (مثلاً) @path کرنے کے لیے standards selectively include میں CLAUDE.md کے package کو استعمال کریں ( @./standards/testing.md )
- files (testing.md, api-conventions.md, deployment.md) .claude/rules/ کو متعدد CLAUDE.md بڑے میں تقسیم کریں

## 3.2 Custom Slash Commands اور Skills بنانا اور Configure کرنا

### Key knowledge:

- `~/ .claude/commands/` بمقابلہ (VCS کے ذریعے shared) **Project commands** میں `~/ .claude/commands/`
- `~/ .claude/skills/` میں SKILL.md frontmatter: `context: fork` , `allowed-tools` , `argument-hint`
- `context: fork` skill کو isolated subagent context سے جدا کرتا ہے تاکہ main session pollute نہ ہو
- Personal skill variants `~/ .claude/skills/` میں رکھے جاسکتے ہیں

### Key skills:

- Project slash commands `.claude/commands/` پوری team میں رکھیں تاکہ پوری
- Verbose output `context: fork` استعمال کریں `skills isolate` والے `کے لیے`
- Skill `allowed-tools` محدود کرنے کے لیے `tools` کے استعمال کردہ
- Required parameters `argument-hint` مانگنے کے لیے

### 3.3 Conditional Convention Loading

#### Path-specific Rules

استعمال کرنا

### Key knowledge:

- `.claude/rules/` files میں YAML frontmatter `paths` تاکہ `glob patterns` شامل ہو سکتا ہے۔ `rules activate` ہوں
- `Path-scoped rules` `matching files edit` وقت `load` کریں، `tokens` اور `context`، `بچتے` ہیں
- `Glob-based path rules` `directory-level CLAUDE.md` میں جب `conventions` سے `بتر` ہو سکتے ہیں جب `tests` (مثلاً) `apply` میں `directories`

### Key skills:

- `paths: ["terraform/**/*"]` `.claude/rules/` files پر load matching files تاکہ صرف `files` کے ساتھ `paths` کے مطابق فائلوں کو تلاش کیا جاسکے۔
- Glob patterns ( `**/*.test.tsx` ) تاکہ `location` استعمال کریں تاکہ `file type` کے لحاظ سے `file type` کے مطابق فائلوں کو تلاش کیا جاسکے۔
- `path-specific` کے بجائے `directory-level CLAUDE.md` بھر میں پھیلی ہوئی `conventions codebase` جب کو ترجیح دیں



## Key skills:

- Interactive input پر hang لے لے کے لیجئے کہ وہ CI میں Claude Code کو -p کے ساتھ چلائیں
- Structured results (مثلاً inline PR comments) کے لیے --output-format json + --json-schema استعمال کریں
- new/unfixed issues (صرف) شامل کریں review results کرتے وقت پہلے run کے بعد دوبارہ commits نہ (کریں report)
- Test generation quality کے لیے CLAUE.md میں testing standards اور available fixtures document کریں
- style نہ ہو اور duplication میں شامل کریں تاکہ context کو existing test files کے وقت tests generate نہ کرے consistent رہے

## 4 Prompt Engineering اور Structured Output (20%)

### 4.1 Accuracy کے ساتھ Explicit Criteria بہتر کرنے کے لیے Design کرنا

#### Key knowledge:

- Explicit criteria vague instructions میں زیادہ مؤثر ہوتے ہیں (مثلاً "flag comments only when they contradict code" بمقابلہ "check comment accuracy")
- "be more conservative" generic guidance concrete categorical criteria کے مؤثر ہیں
- False positives developer trust پر اثر انداز ہوتے ہیں: کچھ categories high false-positive rates میں trust پر بھی categories کم کر دیتی ہیں

#### Key skills:

- Review criteria define کرنا (bugs, security) اور ignore کرنا (minor style) report کرنا
- High false-positive rates والی categories طور پر disable عارضی طور پر
- explicit severity criteria define کے ساتھ code examples کے لیے level پر

### 4.2 Output Consistency کے لیے Few-shot Prompting بہتر کرنے کے لیے استعمال کرنا

#### Key knowledge:

- Few-shot examples consistently formatted, actionable output method پیدا کرنے کا سب سے مؤثر ہے
- Few-shot cases (tool selection, test coverage gaps) کو دکھا سکتا ہے
- Few-shot model کو نہ generalize پر patterns کو نہ defaults میں مدد دیتا ہے، صرف
- Few-shot extraction tasks میں hallucinations کو کم کر سکتا ہے

### Key skills:

- Ambiguous scenarios □ لی □ rationale 4–2 ساتھ □ targeted examples دیں
- Output format (location, issue, severity, suggested fix) □ وال □ examples دکھانے شامل کریں
- میں فرق دکھائیں real issues اور acceptable code patterns □ جو examples ایسے □
- دیں □ examples □ extraction سہ □ درست documents وال □ structures مختلف

### 4.3 Structured Output ☐ ساتھ JSON Schemas اور tool\_use

### Key knowledge:

- `tool_use` with JSON Schemas schema-conformant output اور ختم JSON syntax errors کی ضمانت دینا
  - طریقہ □□ reliable کرنے کا سبب ہے
- `tool_choice: "auto"` میں model text □□ واپس کر سکتا □□؛ "any" لازماً □□ tool call میں اسے □□ کرنی
  - forced selection specific tool □□ منتخب کرتی □□
- Strict JSON Schemas syntax errors مگر semantic errors ختم کرتی □□ ہیں مگر (totals؛ □□ ملتے؛ □□)
  - fields غلط values
- Schema design: required vs optional fields؛ enums □□ ساتھ □□ "other" + detail string extensibility □□ لے □□

### Key skills:

- JSON Schemas کے ساتھ extraction tools define کریں اور results سے data parse کریں
- tool\_choice: "any" یقینی بنانے کے لیے structured output کے وں تو schemas متعدد استعمال کریں
- Specific tool call force کریں: tool\_choice: {"type": "tool", "name": "extract\_metadata"}
- Source information میں values fabricate نہ کی صورت میں fields کے optional/nullable رکھیں
- Extensible categorization کے لیے "unclear" اور "other" enum values + detail fields استعمال کریں

#### 4.4 Extraction Quality ☐ لی ☐ Validation, Retries, اور Feedback Loops Implement کرنا

### Key knowledge:

- Retry-with-error-feedback: correction guide میں concrete validation errors retry prompt کو لے کر شامل کریں
- فائدہ ہوتا ہے کہ retries میں موجود نئی information source اگر
- Feedback loop design: finding والا trigger کو track (detected\_pattern) کرنے
- Semantic errors (totals reconcile نہیں ہوتے) جو syntax errors بمقابلہ tool\_use address نہ ہو سکتے ہیں

### Key skills:

- Original document, incorrect extraction, اور specific validation errors کے ساتھ follow-up prompts دیں
- تو اس کی شناخت کریں (میں) اور (required info صرف external document) کے فوائد کے بغیر retry جب

- False positives analyze `detected_pattern` fields میں findings کرنے کے لیے
- Discrepancies detect `calculated_total` اور `stated_total` extract کرنے کے لیے self-correction design کریں

## 4.5 Efficient Batch Processing Strategies Design کرنا

### Key knowledge:

- Message Batches API: 50% savings, 24-hour processing window, latency SLA guarantee
- Batch processing non-blocking tasks (overnight reports, audits) اور blocking tasks (pre-merge checks) کے لیے مناسب
- Batch API single request میں multi-turn tool calling support
- `custom_id` fields batch request/response correlate کرتی ہیں

### Key skills:

- Blocking checks synchronous API اور overnight/weekly workloads کے لیے Batch API استعمال کریں
- SLA needs مطابق batch submission cadence plan کریں (24-hour guarantee مثلاً 30 processing 4-hour windows کے ساتھ)
- Failed documents (identified by `custom_id`) submit کر کے صرف انہی کو دوبارہ
- Large-scale processing کے ساتھ sample prompts iterate کریں

## 4.6 Multi-instance اور Multi-pass Review Architectures Design کرنا

### Key knowledge:

- Self-review limitations: model reasoning context کو فیصلوں اور اپنے challenge ساتھ رکھتا ہے اور اپنی امکان کم ہوتا ہے
- Independent review instances (generation context بغیر کے) subtle issues میں بہتر ہوتی ہیں
- Multi-pass review: per-file local analysis + cross-file integration pass attention dilution سے بچانا

### Key skills:

- Changes review second independent Claude instance کے بغیر generation context کرنے کے لیے کریں
- Multi-file reviews کو per-file passes + integration passes میں تقسیم کریں تاکہ cross-file dataflow analysis ہو سکے
- Self-rated confidence کے ساتھ verification passes کے reviews استعمال کر کے calibrated سے route طریقہ

## 5 Context Management اور Reliability (15%) ڈومین 5

### 5.1 Critical Information Conversation Context محفوظ رکھنا اور Manage کرنا

#### Key knowledge:

- Progressive summarization کے خطرات: numeric values, percentages, اور dates vague summaries میں بدل جاتے ہیں
- Lost-in-the-middle effect: models long inputs کے شروع اور آخر کو زیادہ reliable سے process طریقہ سے کر سکتے ہیں findings miss ہیں، لیکن درمیان کے
- Tool outputs relevance میں context کے مقابلہ میں disproportionately جمع ہو سکتے ہیں (40+ fields جب 5 fields کے درکار ہوں)
- Subsequent API requests میں full conversation history کی اہمیت

#### Key skills:

- Transactional facts کو summarized history سے باہر ایک persistent “case facts” block میں نکالیں
- Verbose tool outputs کو relevant fields تک trim کریں
- Key findings کو aggregated data کے شروع میں explicit section headings رکھیں
- Subagents کے structured outputs میں metadata (dates, sources) شامل کروائیں

### 5.2 Effective Escalation Patterns Design اور Ambiguity Resolve کرنا

#### Key knowledge:

- Suitable escalation triggers: human کی explicit request, policy gaps/exceptions, اور پانا
- Immediate escalation (explicit request) کے اندر attempt-to-resolve (agent scope) کے مقابلہ میں
- Sentiment analysis اور model confidence self-ratings case complexity کے unreliable proxies ہیں
- Multiple customer matches کے heuristic guessing کے بجائے additional identifiers چاہئیں

#### Key skills:

- System prompt میں explicit escalation criteria few-shot examples کے ساتھ دیں
- Human کی explicit request کو بغیر اضافی investigation کے فوراً execute کریں
- escalate ہو تو silent یا ambiguous کے لیے specific request کسی policy جب
- Tool results میں multiple matches ہوں تو additional identifiers مانگیں



## 5.3 Multi-agent Systems میں Error Propagation Strategies کرنے

### Key knowledge:

- Structured error context (failure type, query, partial results, alternatives) coordinator کو smarter recovery دیتا ہے
- Access failures (timeouts retry decision میں) اور valid empty results (no matches) میں فرق کریں
- Generic error statuses ("search unavailable") coordinator سے valuable context چھپا دیتا ہے
- Silent suppression یا failure پر پورا workflow abort کرنا anti-patterns میں سے ہے

### Key skills:

- Structured error context: failure type, کیا گیا attempt, partial results, possible alternatives
- Access failures کو valid empty results الگ کریں
- Transient failures subagents میں local recovery کریں؛ صرف non-recoverable errors partial results کو propagate کریں
- Synthesis میں coverage annotate کیا اچھی طرح supported اور gaps

## 5.4 Large Codebases Investigate کرتے وقت Context Efficiently Manage کرنے

### Key knowledge:

- Long sessions میں context degradation: model unstable answers اور مخصوص classes دینے لگتا ہے
- Scratchpad files key findings کو context boundaries میں محفوظ رکھتے ہیں
- Subagents کو delegate سے verbose discovery output isolate کرنا
- Structured state persistence crash recovery ممکن بناتی ہے

### Key skills:

- Specific questions subagents spawn کرنا جبکہ main agent high-level coordination میں رکھیں
- Key findings scratchpad files میں refer محفوظ رکھنا اور بعد میں
- Key findings summarize کرنا subagents spawn کرنا
- Long investigations میں context usage کم کرنا `/compact` استعمال کریں

## 5.5 Human Oversight اور Confidence Calibration کے ساتھ Workflows Design کرنے

### Key knowledge:

- Aggregate metrics (97% overall accuracy) specific document types یا fields پر کمزوری چھپا سکتا ہے
- Stratified random sampling high-confidence extractions میں error rates measure کرنا

- Field-level confidence calibration labeled validation sets کے ذریعے
- Automation کے accuracy validate کے حساب سے field segment اور document type سے پر لے کر

### Key skills:

- New error patterns detect کے لیے stratified random sampling implement کریں
- Stable performance validate کے لیے field اور document type کے حساب سے accuracy analyze کریں
- Field-level confidence scores output کے ذریعے labeled data review thresholds calibrate کریں اور
- Low-confidence یا ambiguous-source extractions human review کی طرف route کریں

## 5.6 Multi-source Synthesis میں Provenance Preserve کرنا اور Uncertainty Handle کرنا

### Key knowledge:

- Summarization کے دوران attribution کے "claim → source" mappings رکھو جاتی ہیں اگر
- Aggregation کے دوران structured mappings کے نیچے چاہئیں
- Conflicting statistics کو arbitrarily ایک value کے بجائے attribution منتخب کرنے کے ساتھ conflict annotate کے handle کریں
- Temporal differences کو contradiction کے لیے publication/collection dates سمجھنے سے بچنے کے لیے

### Key skills:

- Subagents کے "claim → source" mappings (URL, document name, quotes) output کروائیں
- Reports کو stable findings اور disputed ones میں الگ structure کریں
- Conflicting values کو annotations کے ساتھ reconciling رکھیں اور
- Correct temporal interpretation کے لیے publication dates شامل کریں
- Content type کے مطابق render کریں: financial data tables، news prose، technical findings، structured lists میں

## Claude Certified Architect — Foundations Certification

### مطالعہ کے گائیڈ (سرکاری امتحانی گائیڈ کی بنیاد پر)

### تعارف

Claude Certified Architect — Foundations سرٹیفیکیشن اس بات کی تصدیق کرتی ہے کہ ایک ماہر Claude-based Claude Code، Claude Agent SDK، Claude API، اور Model Context Protocol (MCP) بنیادی ٹیکنالوجیز جن کے بنیادی علم کو جانچتا ہے — یعنی وہ trade-off حل بنانے کے وقت درست Claude کے ساتھ پروڈکشن ایپلیکیشنز تیار کی جاتی ہیں۔

بنانا agentic systems امتحان کے سوالات حقیقی دنیا کے منظر ناموں پر مبنی ہوتے ہیں: کسٹمر سپورٹ کے لیے، multi-agent research pipelines، Claude Code کو CI/CD شامل کرنا، developer productivity، structured data بنانا، اور غیر ساختہ دستاویزات سے tools نکالنا

## هدف امیدوار

کرتا آپ ship کے ساتھ پروڈکشن ایپلیکیشنز ڈیزائن اور Claude کے جو solution architect مثالی امیدوار ایک کے پاس کم از کم 6 ماہ کا عملی تجربہ ہونا چاہیے:

- Claude Agent SDK** — multi-agent orchestration، subagents کو delegate کرنا، tool integration، lifecycle hooks
- Claude Code** — CLAUDE.md، MCP servers، Agent Skills، planning mode
- Model Context Protocol (MCP)** — backend integration کے لیے tools اور resources
- Prompt engineering** — JSON schemas، few-shot examples، data extraction templates
- Context windows** — لمبی دستاویزات، multi-agent context passing
- CI/CD pipelines** — automated code review، test generation
- Escalation and reliability** — error handling، human-in-the-loop

## امتحانی فارمیٹ

| پیرامیٹر         | قدر                                      |
|------------------|------------------------------------------|
| سوال کی قسم      | Multiple choice (میں سے 1 درست 4)        |
| اسکورنگ          | 100–1000 scale، passing score <b>720</b> |
| Guessing penalty | نہیں (سوال کا جواب دیں!)                 |
| Scenarios        | 8 4 (randomly selected) میں سے           |

## امتحانی مواد: 5 ڈومینز

| ڈومین                                       | وزن        |
|---------------------------------------------|------------|
| 1. Agent architecture and orchestration     | <b>27%</b> |
| 2. Tool design and MCP integration          | <b>18%</b> |
| 3. Claude Code configuration and workflows  | <b>20%</b> |
| 4. Prompt engineering and structured output | <b>20%</b> |
| 5. Context management and reliability       | <b>15%</b> |

# امتحانی منظرنامہ

## Scenario 1: Customer Support Agent

returns, billing disputes, account issues استعمال کرتے ہوئے Claude Agent SDK بناتے ہیں جو agent آپ ایک agent MCP tools (get\_customer, lookup\_order, process\_refund, escalate\_to\_human) سنبھالتا ہے۔ مناسب first-contact resolution % استعمال کرنا 80 دفعہ ساتھ۔

## Scenario 2: Claude Code ساتھ Code Generation

code generation, refactoring, debugging, documentation آپ کو اسدے custom slash commands اور CLAUDE.md configuration استعمال کرنا، اور integrate

## Scenario 3: Multi-Agent Research System

tasks کو coordinator agent specialized subagents سونپتا: web research, document analysis, synthesis, report generation اور citations سسٹم کو

## Scenario 4: Developer Productivity Tools

agent engineers کو unfamiliar codebases explore، boilerplate code اور routine tasks MCP servers اور Built-in tools (Read, Write, Bash, Grep, Glob) میں مدد دیتا ہے۔

## Scenario 5: Claude Code for Continuous Integration

pull اور automated code reviews, test generation, CI/CD pipeline کو Claude Code false positives ایسے بندے چاہئیں کہ Prompts حاصل ہو۔

## Scenario 6: Structured Data Extraction

JSON schemas کو output، سسٹم غیر ساختہ دستاویزات سے معلومات نکالتا ہے۔ درست طور پر سنبھالنے چاہئیں edge cases اور high accuracy اسدے

## Scenario 7: Conversational AI Architecture Patterns

multi-turn conversational systems میں جن میں context window management، turns متعدد، memory strategies، execution محفوظ، tool design، اور مہم یا متضاد، instructions میں user inputs کو سنبھالنا شامل ہے۔

## Scenario 8: Agentic AI Tools (ہماری مدد کریں — مواد موجود نہیں)

میں شامل نہیں دی لیکن ابھی تک اس candidates کی اطلاع امتحان دینے والے scenario اس میں share میں GitHub Issues کے سوالات دیکھیں، تو انہیں scenario اگر آپ نے اصل امتحان میں اس شامل کر سکیں آپ کا تعاون اس شخص کی مدد کرے گا جو امتحان کی تیاری کر coverage تاکہ ہم مکمل رہا

# نظریاتی بنیادیں: حصہ 1

یہ حصہ و بنیادی نظریہ بیان کرتا ہے جس کی آپ کو امتحان میں کامیابی کے لیے ضرورت ہے۔ مواد کو topic کے مطابق — اس کے domains کے مطابق ترتیب دیا گیا ہے، نہ کہ امتحانی concepts اور technologies کی گہری سمجھ بھنی کے لیے۔

## کی بنیادیں Claude API — Model Interaction: باب 1

دستاویزات: [Messages API](#) | [Prompt Engineering](#)

### 1.1 API Request Structure

Claude API میں عموماً یہ Claude Messages API request پر کام کرتا ہے request-response model ایک request-response model میں شامل ہوتا ہے:

```
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 1024,
  "system": "You are a helpful assistant.",
  "messages": [
    {"role": "user", "content": "Hi!"},
    {"role": "assistant", "content": "Hello!"},
    {"role": "user", "content": "How are you?"}
  ],
  "tools": [...],
  "tool_choice": {"type": "auto"}
}
```

م فیلڈز:

- `model` — model selection ( `claude-opus-4-6`, `claude-sonnet-4-6`, `claude-haiku-4-5` )
- `max_tokens` — response میں زیادہ سے زیادہ tokens
- `system` — system prompt (model کی تعریف)
- `messages` — conversation history (full history کے لیے ضروری)
- `tools` — available tools کی definitions
- `tool_choice` — tool selection strategy

### 1.2 Message Roles

roles میں تین array: `messages`

- `user` — user messages
- `assistant` — model responses (history کے لیے ضروری)

- `tool` — tool call results (عملاً `tool_result` content block)

محفوظ state کے درمیان Model requests بھیجیں conversation history میں مکمل API request م بات: ر آزاد ہوتی call نہیں رکھتا؛ ر

## 1.3 stop\_reason

Claude API response میں `stop_reason` کے جو بتاتا ہے کہ:

| Value           | Description              | Action                                 |
|-----------------|--------------------------|----------------------------------------|
| "end_turn"      | Model جواب مکمل کر لیا   | کو نتیجہ دکھائیں user                  |
| "tool_use"      | Model tool call چاہتا ہے | واپس دیں result چلائیں اور tool        |
| "max_tokens"    | Token limit ختم ہو گئی   | response truncated؛ limit میں          |
| "stop_sequence" | Stop sequence مل گئی     | کریں handle کے مطابق application logic |

کرتے loop control ہیں — یہی signals سب سے اہم "end\_turn" اور "tool\_use" میں Agentic systems ہیں

## 1.4 System Prompt

System prompt متعین کرتا ہے behavioral rules اور context کے جو instruction ایک خاص

- `messages` array الگ ہوتا؛ الگ `system` field کا حصہ نہیں ہوتا؛
- user messages پر فوقیت رکھتا ہے
- بار بار پوری گفتگو میں لاگو رہتا ہے load ایک بار
- کی وضاحت کے لیے استعمال ہوتا ہے output format اور role, constraints,

کو غیر ارادی طور پر متاثر کر سکتی ہے مثال tool selection wording کی system prompt: امتحان کے لیے اہم ضرورت سے زیادہ `get_customer` کو model کی تصدیق کریں "جیسی ہدایت customer کے طور پر" ہمیشہ استعمال کرنے پر لے جا سکتی ہے

## 1.5 Context Window

Context window اس میں شامل process ایک وقت میں model کے جس (tokens) و کل متن ہے:

- System prompt
- message history مکمل
- Tool definitions
- Tool results

اہم مسائل:

1. **Lost-in-the-middle effect:** درمیان کی input لمبے، مؤثر ہوتی ہیں، آغاز اور اختتام کی معلومات زیادہ مؤثر ہوتی ہیں، حل: اہم معلومات شروع یا آخر میں رکھیں details miss

2. **Tool results** واپس tool 40+ fields شامل کرتی ہیں اگر output میں tool call context کا جمع ہونا: ہر Tool results ضائع ہوتا ہے context کو مگر صرف 5 امیون تو
3. **Progressive summarization:** history compress کرنے پر numeric values, percentages, اور dates اکثر vague ہوتی ہیں

## 2 tool\_use اور Tools: باب 2

Tool Use: دستاویزات

### 2.1 tool\_use کیا ہے

براہ راست Model کو external functions call کرنے کے لیے Claude جس کے ذریعے mechanism tool\_use واپس کرتا ہے result کو execute اس کے لیے code بنانا ہے: آپ کا structured tool call نہیں چلاتا — وہ code درست ہے

### 2.2 Tool Definition

tool JSON schema کے ذریعے define ہوتا ہے:

```
{
  "name": "get_customer",
  "description": "Finds a customer by email or ID. Returns the customer profile, including name, email, order history, and account status. Use this tool BEFORE lookup_order to verify the customer's identity. Accepts an email (format: user@domain.com) or a numeric customer_id.",
  "input_schema": {
    "type": "object",
    "properties": {
      "email": {"type": "string", "description": "Customer email"},
      "customer_id": {"type": "integer", "description": "Numeric customer ID"}
    },
    "required": []
  }
}
```

Tool description نکات:

1. Minimal descriptions کا mechanism tool selection کی سبب بنتی غلط ہے
2. Description میں شامل کریں: input format، کیا کرتا ہے، کیا واپس کرتا ہے tool، اور یہ کب استعمال ہوتا ہے
3. Overlapping descriptions سے بچیں
4. Built-in tools اور MCP tools overlap کو ہٹانے کے لیے MCP tool واضح بنائیں

## 2.3 tool\_choice

| Value                                                       | Behavior                         | کب استعمال کریں              |
|-------------------------------------------------------------|----------------------------------|------------------------------|
| <code>{ "type": "auto" }</code>                             | Model خود ط کرتا ہے              | default عام                  |
| <code>{ "type": "any" }</code>                              | Model tool call کو لازماً کرے گی | structured output جب چاہیے   |
| <code>{ "type": "tool", "name": "extract_metadata" }</code> | Model tool call کو لازماً کرے گی | forced first step / order ہے |

## 2.4 JSON Schemas for Structured Output

reliable لینے کا سب سے structured output Claude استعمال کرنا `tool_use` کے ساتھ JSON schemas کی semantic correctness نافذ کرتا ہے، مگر required structure کم کرتا ہے اور syntax errors طریقہ سے یقیناً ضمانت نہیں دیتا

### اصول Design:

- رکھیں جو ہمیشہ ملنے چاہئیں required fields صرف وہ
- استعمال کریں nullable fields کے لیے missing data ممکن
- ضائع نہ ہو information شامل کریں تاکہ enum values جیسے "unclear" اور "other"

## 2.5 Syntax اور Semantic Errors

| Error type | Example                                 | Mitigation                                |
|------------|-----------------------------------------|-------------------------------------------|
| Syntax     | Invalid JSON, field type غلط            | JSON schema کے ساتھ <code>tool_use</code> |
| Semantic   | Totals match نہیں کرتے، value field غلط | Validation checks, feedback retry         |

# بنانا Claude Agent SDK — Agentic Systems: باب 3

دستاویزات: [Agent SDK](#) | [Hooks](#) | [Subagents](#) | [Sessions](#)

## 3.1 Agentic Loop

چلانا sequence کی actions صرف جواب نہیں دیتا، بلکہ Claude میں Agentic loop

- Claude کو tools کے ساتھ request بھیجیں
- Response لیں
- چیک کریں `stop_reason`:
  - "tool\_use" → tool چلائیں، result history پھر دوبارہ request میں ڈالیں



- "end\_turn" → task کو دین user مکمل، نتیجہ

مکمل ہونے تک دہرائیں۔ 4.

Text parsing یا arbitrary iteration دیکھیں کہ `stop_reason == "end_turn"` کی لیے `signal: completion` درست قابلِ اعتماد نہیں ہے۔

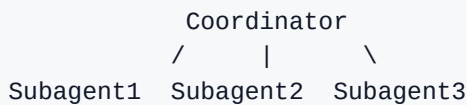
## 3.2 AgentDefinition

```
agent = AgentDefinition(
    name="customer_support",
    description="Handles customer requests for returns and order issues",
    system_prompt="You are a customer support agent...",
    allowed_tools=["get_customer", "lookup_order", "process_refund",
"escalate_to_human"],
)
```

اصول اپنائیں کہ Least privilege کے allowed\_tools اور، system\_prompt، description، name: کم چیزیں

## 3.3 Coordinator اور Subagents

استعمال کرتی ہیں hub-and-spoke topology اکثر Multi-agent systems:



Coordinator task کو توڑتا ہے، subagents نتائج جوڑتا ہے، کام سونپتا ہے، منتخب کرتا ہے، errors handle کرتا ہے، جواب پہنچاتا ہے۔ final تک user، اور

نہیں کرتے۔ inherit خود بخود parent history الگ ہوتا ہے؛ وہ context کا subagents: کم اصول

## 3.4 Task Tool

Subagents Task tool کے ذریعے spawn کیے جاتے ہیں۔ Coordinator کے allowed\_tools میں "Task" ہونا چاہیے۔

context passing: صحیح

- Task: "Analyze the document" کافی نہیں
- Task: "Analyze this document. Full text: ... Output schema: ..." درست ہے

ممکن ہے کہ parallel subagents بھیج سکتا ہے، لہذا Task s multiple میں response ایک Coordinator

## 3.5 Hooks

کرتے ہیں کہ work پر points کے مخصوص lifecycle Hooks

PostToolUse tool result کو model پر لے کر normalize تک پہنچانے سے پہلے:

```
@hook("PostToolUse")
def normalize_dates(tool_result):
    return normalized_result
```

PreToolUse policy violation block کر سکتا ہے:

```
@hook("PreToolUse")
def enforce_refund_limit(tool_call):
    if tool_call.name == "process_refund" and tool_call.args.amount > 500:
        return redirect_to_escalation(tool_call)
```

Critical business rules کو دستی ہے۔ prompt instructions probabilistic ہیں؛ hooks deterministic ہیں: یاد رکھیں rules کو لے کر hooks ہیں۔

## باب 4: Model Context Protocol (MCP)

[MCP](#) | [Tools](#) | [Resources](#) | [Servers](#) : دستاویزات

### 4.1 MCP کیا ہے

MCP resource سے جوڑتا ہے اس کے تین بنیادی Claude کو external systems کو جو open protocol ہے:

1. **Tools** — actions کے لیے functions
2. **Resources** — context data (documentation, schemas, catalogs)
3. **Prompts** — predefined task templates

### 4.2 MCP Servers

MCP server ایک process جو tools/resources expose کرتا ہے Connect پر tools discover ہونے پر۔ مطابقت استعمال ہونے والے descriptions ہیں اور

### 4.3 Configuration

Project config ( `.mcp.json` ) لے کر:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {"GITHUB_TOKEN": "${GITHUB_TOKEN}"}
    }
  }
}
```

User config سہ آتی ہے secrets environment variables میں ہوتی ہے؛ Project config version control (~/claude.json) personal/experimental servers کے لیے ہوتی ہے

## 4.4 isError

واپس کریں structured error کے بجائے Generic errors استعمال کریں `isError: true` میں MCP tool errors

```
{
  "isError": true,
  "content": {
    "errorCategory": "transient",
    "isRetryable": true,
    "message": "The service is temporarily unavailable."
  }
}
```

## 4.5 Resources

Resources کے پڑھ سکتا ہے action بغیر agent میں جو data و Resources: schemas, docs, content catalogs, summaries، exploratory tool calls میں کم کرتے ہیں

# 5 اور Claude Code — Configuration Workflows

دستاویزات: [Claude Code](#) | [Memory / CLAUDE.md](#) | [Skills](#) | [MCP](#) | [Hooks](#) | [Sub-agents](#) | [GitHub Actions](#) | [Headless](#)

## 5.1 CLAUDE.md Hierarchy

تین سطحوں پر ہو سکتا ہے CLAUDE.md

- **User-level:** `~/.claude/CLAUDE.md` — پر لاگو user صرف اسی
- **Project-level:** `.claude/CLAUDE.md` یا `root` `CLAUDE.md` — کا لی team پوری
- **Directory-level:** subdirectories میں `CLAUDE.md` — folder conventions مخصوص

## 5.2 @path Syntax

CLAUDE.md میں @path کا external files refer کا تاکہ جا سکتا ہے میں تاکہ:

Coding standards `@./standards/coding-style.md` میں ہیں  
Test rules `@./standards/testing-requirements.md` میں ہیں

## 5.3 .claude/rules/

`.claude/rules/` rules کو topic کا لحاظ سے organize کا کرتا ہے اور `paths` کا ذریعہ conditional loading دیتا ہے:

```
---
paths: ["src/api/**/*"]
---
```

استعمال کریں explicit error handling اور async/await کہ لیں API files

## 5.4 Commands اور Skills

`.claude/commands/` project commands کا لی، جبکہ `~/.claude/commands/` personal commands ملتی ہیں۔ skills کا ساتھ frontmatter میں `.claude/skills/` کا لی

## 5.5 Skills اور context: fork

`context: fork` skill کو isolated context میں چلاتا ہے `allowed-tools` اور tool access کا `argument-hint` مانگیں required input کا

## 5.6 Planning Mode

Planning mode کا لی، بہترین unfamiliar codebases اور multiple approaches، تبدیلیوں architectural بڑی Direct execution کا لی، بہتر changes چھوٹے اور واضح

## 5.7 /compact اور /memory

`/compact` context compress کرتا ہے، اور `/memory` notes اور preferences میں مدد دیتا ہے





## 9 Escalation اور Human-in-the-Loop: باب 9

### 9.1 Escalate کب کریں

Explicit user request, policy gaps, progress نہ ہونا, threshold سے اوپر financial action, یا ambiguous matching میں Sentiment analysis اور self-rated confidence unreliable ہیں۔ Escalate کی صورت میں

### 9.2 Escalation Patterns

human کو hand off نہ کرنا یا explicit insistence یا policy gap نہ کرنے کی کوشش کریں، پھر اگر resolve نہ ہو، مسئلہ حل کریں

### 9.3 Structured Handoff

human شامل کریں تاکہ recommended action اور، actions taken، issue summary، customer ID پر Escalation ملا۔ context کو مکمل operator

### 9.4 Confidence Calibration

Field-level confidence scores، calibration on labeled data، اور stratified sampling استعمال کریں

## 10 Error Handling میں Multi-agent Systems: باب 10

### 10.1 Error Categories

Transient errors retryable ہیں، validation errors input fix ہیں، مانگتی business errors policy-driven ہیں۔ کی جاتی ہیں۔ اور permission errors generally escalate ہیں، اور

### 10.2 Anti-patterns

Generic “search unavailable” response، silent suppression، workflow abort یا پورا کرنا نقصان دہ۔ Structured errors واپس کریں

### 10.3 Structured Subagent Error

Structured error میں failure type، attempted query، partial results، alternative approaches، اور coverage شامل کریں۔ impact

## 10.4 Final Synthesis میں Coverage

پارٹیاں، اور کچھ partially covered ہیں، کون سی sections fully covered میں واضح کریں کہ کون سی Report باقی ہیں۔ gaps

# 11 Context میں Production Systems: باب 11 Management

## 11.1 Facts Separate Block میں

Transactional facts کو persistent case facts block تاکہ summarization میں رکھیں تاکہ critical data کے باوجود زچہ و

## 11.2 Tool Results Trimming

PostToolUse hook رکھیں relevant fields سے صرف

## 11.3 Position-aware Input

Important findings اور action items شروع میں اور lost-in-the-middle effect آخر میں رکھیں تاکہ

## 11.4 Scratchpad Files

Verbose findings scratchpad تاکہ long investigations میں لکھیں تاکہ context محفوظ رہے

## 11.5 Subagents کو Delegate کرنا

Exploration subagents میں اور main context میں صرف summary رکھیں

## 11.6 Structured State Persistence

ممکن ہے crash recovery تاکہ export میں JSON file یا state manifest اپنی agent سے

# 12 Provenance رکھنا: باب 12 کو محفوظ رکھنا

## 12.1 Attribution Loss

Claim رکھیں confidence اور، source URL، publication date کے ساتھ



## 12.2 Conflicting Data

کریں conflict flag ساتھ محفوظ رکھیں اور attribution دیں تو دونوں کو values مختلف sources اگر

## 12.3 Dates

دیکھیں publication dates سمجھنا سہل ہے contradiction کو Temporal differences

## 12.4 Content Type مطابق Render

Financial data tables میں، news prose میں، technical findings structured lists اور time series میں دیکھائیں chronological order

# 13 Claude Code Built-in Tools: باب

## 13.1 Tool Selection Reference

| Task                      | Tool  | Example                      |
|---------------------------|-------|------------------------------|
| تلاش کرنا file سہ pattern | Glob  | <code>**/*.test.tsx</code>   |
| file contents میں search  | Grep  | function name, error message |
| پڑھنا file                | Read  | analysis کو لے               |
| بنانا file نیا            | Write | scratch سہ                   |
| precise edit              | Edit  | unique text match            |
| shell command             | Bash  | git, npm, tests              |

## 13.2 Incremental Investigation

دیکھیں consumer files پھر، usages پھر، entry points نہ پڑھیں؛ لہذا files ایک ساتھ سب

## 13.3 Read + Write Fallback

کریں Write کریں، پھر programmatically کریں، تبدیلی Read کو تو Edit fail اگر

## امتحانی ڈومین نوٹس: II حصہ



- C) Tool description agent شامل کریں جن میں few-shot examples اور instructions میں تفصیلی `dry_run=true` کو `call` سے `replace` کریں `preview_remove_member` impact details اور single-use confirmation token کو `preview` سے `bind`، درکار token کو `execute_remove_member` واپس کرتا ہے؛ `execute_remove_member` [درست] کرتا ہے

D) Token-binding approach سے `architecturally` `preview` ناممکن ہو جاتا ہے `execute tool` کو `literally` `execute` — `execute tool` جو صرف `token` `generate` `preview tool` `orchestration infrastructure` یا (A)، `timing heuristics` (C)، `LLM instructions` جو `approach` `code level` پر `constraint enforce` پر انحصار کے بجائے (B) کرتی ہے

## سوال 62

`search_catalog` tool 12% fail وقت `Production monitoring` کا آپ کا `search_catalog` سے معلوم ہوتا ہے کہ آپ کا `search_catalog` `query syntax errors` پر کامیاب ہوتا ہے، اور `retry 4` میں جو `network timeouts` 8% `error types` return یکساں طریقہ سے `error types` کامیاب نہیں ہوتا ہے فی الحال دونوں `retries` میں ہوتی ہیں

کریں گے؟ `modify` کیسے `error handling` کی tool آپ

- A) System prompt میں few-shot examples شامل کریں جو `network errors` میں فرق کیسے کریں `syntax errors`
- B) `uniformly exponential backoff retry logic` apply کریں `errors` پر تمام
- C) Tool `network timeouts` اندر `automatic retry with backoff` implement کریں؛ `syntax errors` کے ساتھ فوری واپس کریں [درست] `parameter validation details`
- D) `retryable` boolean flag اور `error type details` کو `errors` تمام

C) Tool level پر `transient errors` `retries` handle کرنا صحیح ہے `abstraction boundary` — tool `error type` پر `agent` کر سکتا ہے بغیر `deterministic retry logic` کے بارے میں یقینی علم اور `error type` `Uniform backoff` (B) `syntax errors` `prompt instructions` (A) follow `interpret` (D) `flag` انحصار کے `errors` پر وقت ضائع کرتا ہے جو کبھی کامیاب نہیں ہوتے

## سوال 63

بہت کم "risk tolerance" اور "میری user"، میں `turns` کی گفتگو کے کئی `Investment strategy` `returns maximize` پھر "میں اپنا" کرنا چاہتا ہے؟ `invest` کرنا چاہتا ہوں اب وہ پوچھتا ہے: "مجھے کس چیز میں `returns maximize` پھر "میں اپنا"

سے `priority` کی اصل `recommendation user` سب سے بہتر طریقہ سے یقینی بنانا ہے کہ `approach` کونسا ہے؟

- A) پوچھیں کہ کونسی چیز زیادہ اہم ہے [درست] `user` تضاد کو سامنے لائیں اور
- B) `recommendations` کے لیے الگ الگ `scenarios` دونوں
- C) `preference` سب سے حال میں بیان کی گئی
- D) `balanced portfolio` تضاد کو حل کیے بغیر

مانگنا کی واحد clarification برا راست متضاد ہوں، تو تضاد سامنے لانا اور user کی preferences: جب A low risk کرنا اور Returns maximize کے اصل ارادہ سے ہم آہنگ ہو۔ user recommendation طریقہ کے بنیادی طور پر ناقابل مطابقت ادا ف میں جن کے لیے انسانی فیصلہ درکار ہے tolerance

## سوال 64

jazz کے "مجھے User کرتے ہیں playlist preferences refine میں conversation turns کئی Users: صورتحال "پسند ہیں؟ genres پوچھتا ہے "آپ کو کون سے Claude، بعد messages پسند ہے" کے دو

سب سے زیادہ ممکنہ وجہ کیا ہے؟

- A) Claude کو conversation memory کے لیے vector database connection برقرار رکھنے کے درکار ہے
- B) Model کی context window exceed ہو گئی ہے
- C) Claude API کو session\_id parameter کے درکار ہے
- D) شامل نہیں کر رہی ہے [درست] messages میں بچھلے messages array application آپ کی

request کے stateless API call ہیں — server-side memory کے پاس Claude: کیوں D کا کوئی علم turns کے پاس بچھلے Claude، شامل کیے بغیر conversation history میں مکمل messages array کا حصہ نہیں ہیں؛ دو architecture کے Claude (C) session\_id اور (A) Vector databases ہیں ہوتا ناممکن (B) context window overflow کے لیے exchange کے messages

## سوال 65

History تک پہنچ گئی ہے conversation 78,000 tokens، بعد cooking session صورتحال: 40 منٹ کے شامل ہیں آپ کو اسم معلومات discussion اور عمومی، allergies, recipe scaling, clarified cooking terms، کم کرنے ہیں tokens محفوظ رکھتے ہوئے

کے درمیان بہترین توازن رکھتا ہے؟ token reduction اور approach preservation کونسا

- A) خلاصہ کریں conversation history پوری
- B) رکھیں tokens صرف سب سے حالیہ 20,000
- C) خلاصہ کریں، اور discussion نکالیں، عمومی (allergies, quantities, preferences) structured data اسم رکھیں [درست] verbatim کو exchanges حالیہ
- D) کریں retrieve کے ذریعہ متعلقہ حصہ semantic search کریں اور conversation externally store مکمل

Allergies معلومات محفوظ رکھتی ہے valuable پر سب سے زیادہ cost سب سے کم Hybrid approach: کیوں C (summarization) ہوتا ہے میں compact structured block ایک critical facts جیسے recipe quantities اور exchanges ہوتی ہے، اور حالیہ discussion summarize عمومی، (سہ بچاتے ہوئے precision loss کے دوران conversational coherence کے لیے verbatim میں Options A اور B dietary information اسم کے لیے ضرورت سے زیادہ پیچیدہ ہے cooking session ایک D ہیں؛

## سوال 66

ka preferences اور topics پر assistant کے conversations میں reports کرتے ہیں کہ طویل Users: صورتحال رکھتی ہے message pairs صرف آخری 25 implementation حساب کھو دیتا ہے آپ کی موجود

کیا ہے؟ solution سب سے مؤثر

- A) Hybrid approach: messages پرانے حالیہ messages کا خلاصہ کرتے ہوئے حالیہ messages پرانے
- B) vector similarity search پر conversation history مکمل
- C) Window 50 message pairs تک بڑھائیں
- D) آگے جوڑیں running summary کا خلاصہ کریں اور dropped messages میں turn

A کیوں: Hybrid approach کے دونوں پہلوؤں کو حل کرتی ہے exact recent context کی (کے لیے) (conversational coherence) برقرار compressed representation کی preferences جبکہ پرانے کی (کے لیے) (miss کو context میں) (B) Vector search صرف اسی مسئلہ کو ملتوی کرتا ہے (C) بڑھانا Window رکھتی ہے (D) semantically سے current query کر سکتی ہے جو

## سوال 67

بڑھ جاتی latency سے زیادہ ہوتے ہیں تو 50 turns conversations reports کرتے ہیں کہ جب Users: صورتحال بھی costs اور

بنیادی وجہ کیا ہے؟

- A) شامل ہوتی ہے conversation history کے ساتھ پوری API request
- B) Model gradually responses generate کرتا ہے
- C) History سے جو جاتا ہے database operations کے ساتھ
- D) Model بناتا ہے internal user profile کی ضرورت والا processing زیادہ

A کیوں: Claude کی API پر stateless مکمل طور پر request کو messages array میں پوری request بڑھتی ہے، conversation history شامل کرنی ہوتی ہے جیسے جیسے conversation history کے درمیان کوئی Model calls دونوں بڑھاتا ہے cost اور directly processing latency ل جاتی ہے، جو internal state (D غلط ہے)

## سوال 68

تک بڑھ گئی ہے جب conversation history 85,000 tokens، بعد weekly sessions: صورتحال: تین مہینوں کے discussions پچھلی assistant کے بارے میں کیا نتیجہ اخذ کیا تھا؟، تو theme کے isolation پوچھتا ہے "م نہ user کرنے کے بجائے عام جوابات دیتا ہے reference

کیا ہے؟ approach سب سے مؤثر

- A) Rolling window truncation
- B) Key conclusions capture کے ہوئے progressive summarization
- C) Relevant exchanges retrieve کے لیے (درست) semantic embeddings کرنے کے لیے
- D) Discussion conclusions mark کے لیے structured XML tags کرنے کے لیے

**C scale** discussion کو جو تین مہینوں کی approach واحد semantic search پر conversation history پر **کیوں:**

Rolling window کر سکتی surface specific relevant exchanges demand پر کر سکتی جبکہ truncation (A) history کو Progressive summarization (B) discussions کا بیشتر حصہ ضائع کر دے گی۔

پوچھنے والے users کھو دیتی ہیں جن کے بارے میں specific conclusions، کرتی ہے compress میں abstractions

## سوال 69

کرتا ہے، system prompt guidelines follow میں 10-15 Claude، دوران QA testing: صورتحال  
 اندر token limits ابھی Conversation آجائی ہے deviation میں responses لیکن بعد کے

کیا ؟ solution بہترین

- A) Behavioral guidelines میں منتقل کریں □ user message کو پلا □
- B) 20 turns conversation □ شروع کریں □ بعد نئی
- C) Conversation breakpoints پر guidelines reinforce □ وال □ user-role messages insert [درست]
- D) Non-compliant responses کو regenerate □ لپ □ post-response validation □ استعمال کریں □

history کا براہ راست مقابلہ کرتا ہے Behavioral reminders کا periodic injection instruction drift کو regular intervals پر constraints re-establish کرتا ہے Guidelines کو user کو پر Accumulate message کو Post-response validation (D) corrective میں منتقل کرنا authority ان کا (A) preventive اور significant latency شامل کرتی ہے

## سوال 70

□□ teaching methodology اور adaptation rules □□ جو AI tutor 2,800 token-system prompt میں **صورتحال:** آپ کو define 12 □□□ □□□ assistant proficiency levels □□□ کر دیتا □□، بعد 12 turns کرتا □□□

کیا fix سب سے مؤثر

- A) 5–4 turns میں inject کریں
- B) Verbose rules کو proficiency-level adaptation demonstrate کرنے والے few-shot examples سے replace کریں [درست]
- C) Critical rules کو system prompt کے آخر میں رکھیں
- D) Responses evaluate کریں اور اگر difficulty level match نہ ہو تو regenerate کریں

**B کیوں:** Declarative rules 2,800 والا token system prompt drift کیونکہ abstract rules vulnerable لی concrete few-shot examples کو Verbose rules کا تقاضا کرتے ہیں reasoning model میں turn ر clear کرنا لی match کو model ، کریں demonstrate proficiency-level adaptation correct کرنا جو replace ہوتا ہے reliably follow سے زیادہ abstract instructions میں ، turns دیتا ہے — ، کئی behavioral patterns

## سوال 71

clarifying کرنا، اور reasoning explain برقرار رکھنا، اپنی enthusiastic tone کو assistant **صورتحال**: آپ کے □  
□ ہونی چاہئیں؟ define □ اس behavioral guidelines یوجہ □ ہیں □ □ questions



## سوال 74

Assistant 4+ clarifying requests میں "جیس" venue book اکثر "پارٹی ک" لیں Users: صورتحال  
abandonment % پوچھتا ، جس سے 35 questions

کرتا ؟ improve کو بہترین طریقہ سے approach trade-off کونسا

- A) Hidden defaults کے ساتھ آگے بڑھیں
- B) میں پوچھیں compound message ایک clarifying questions تمام
- C) Assumptions explicitly بیان کریں اور [درست] corrections بڑھیں
- D) Structured intake form استعمال کریں

دیتا جبکہ غلط response کو فوری، مفید user بیان کرنا اور آگے بڑھنا Assumptions explicitly: کیوں C  
کو نہیں بتاتا کیا user (A) Hidden defaults کرنے کی صلاحیت برقرار رکھتا correct کو assumptions  
Structured کا تقاضا کرتی upfront effort سے user پھر بھی (B) Compound question list کیا گیا assume  
form (D) کی بجائے زیادہ شامل کرتا friction کم

## سوال 75

turns rules follow استعمال کرتا ابتدائی assistant contractor-persona system prompt صورتحال: آپ کا  
Conversational length 2,500 tokens صرف دیتا assistant generic advice تک 7 turn کرتے ہیں، لیکن

سب سے زیادہ ممکنہ وجہ کیا ہے؟

- A) System prompts کے لیے initial behavior establish صرف
- B) Turns accumulate کے ساتھ model کے نوٹ کی attention کمزور ہوتی
- C) Accumulated assistant responses system prompt اثر کو dilute [درست] میں
- D) System prompt صرف ایک بار بھیجا جاتا ہے

System prompt جمع ہوتے ہیں assistant responses میں conversation history کیوں: جیس جیس C  
behavioral constraints کو reflect والے text کا proportion بڑھتا ہے assistant-generated content کے  
بجائے اپنے پائل system prompt instructions پر بھی token lengths چھوٹے Model کم ہوتا جاتا ہے relative  
outputs کرتا compound کو drift، کرتا follow کو زیادہ patterns کے

## سوال 76

کئی Assistant "میں مدد کر سکتے ہیں؟ report کرتے ہیں جیس" کیا آپ requests میں Users: صورتحال  
abandonment % جس سے 40، (کیا؟ deadline؟ کیا مدد؟ report؟ کون سی) کرتا respond سوالات پوچھ کر  
ہوتا

کیا solution بہترین

- A) Reasonable assumptions کی پیشکش کریں [درست] adjustments بیان کریں، اور explicitly کریں، انہیں
- B) Respond کے ساتھ smaller model کرنے سے پائل ambiguity classify کریں
- C) Assumptions predefined interpretations بیان کیے بغیر استعمال کریں
- D) Assistant تک محدود کریں clarifying question میں ایک turn کو



رکھتے ہیں اور in control کو user کو informed کے ساتھ آگے بڑھنا Reasonable stated assumptions: کیوں A کرتی ہے confuse کو users (C) Silent predefined interpretations کو مکمل طور پر ختم کرتا ہے back-and-forth کی back-and-forth turns بھی (D) One-question limit ہے، match ان کے ارادے سے response میں جب latency اور infrastructure مسئلہ حل کے بغیر core UX (B) Smaller classification model ضرورت ہے شامل کرتا ہے complexity

## عملی مشقیں

### Exercise 1: Multi-tool Agent with Escalation Logic

Goal: tool integration, structured errors, اور escalation کے ساتھ agent loop بنائیں

### Exercise 2: Configuring Claude Code for Team Development

Goal: CLAUDE.md, custom commands, path-specific rules, اور MCP servers configure کریں

### Exercise 3: Structured Data Extraction Pipeline

Goal: JSON schema, validation/retry loops, اور Message Batches API کے ساتھ extraction pipeline بنائیں

### Exercise 4: Designing and Debugging a Multi-agent Research Pipeline

Goal: subagent orchestration, explicit context passing, error propagation, اور source tracking implement کریں

## Appendix: Technologies and Concepts

| Technology       | Key aspects                                                                                                            |
|------------------|------------------------------------------------------------------------------------------------------------------------|
| Claude Agent SDK | agent loops, <code>stop_reason</code> , hooks, <code>Task</code> , <code>allowedTools</code>                           |
| MCP              | servers, tools, resources, <code>isError</code> , <code>.mcp.json</code>                                               |
| Claude Code      | CLAUDE.md, <code>.claude/rules/</code> , <code>.claude/commands/</code> , <code>.claude/skills/</code> , planning mode |
| Claude Code CLI  | <code>-p</code> , <code>--output-format json</code> , <code>--json-schema</code>                                       |
| Claude API       | <code>tool_use</code> , <code>tool_choice</code> , <code>stop_reason</code> , <code>max_tokens</code>                  |

|                        |                                                     |
|------------------------|-----------------------------------------------------|
| Message Batches API    | 50% savings, 24-hour window, <code>custom_id</code> |
| JSON Schema            | required/optional, nullable, enum values            |
| Few-shot prompting     | ambiguous scenarios ❌ ❌ examples                    |
| Prompt chaining        | sequential decomposition                            |
| Context window         | token budget, lost-in-the-middle                    |
| Confidence calibration | field-level scoring, stratified sampling            |

## Out-of-Scope Topics

❌: موضوعات امتحان میں شامل نہیں ہیں

- Claude models کی fine-tuning
- Authentication, billing, یا account management
- implementation کی تفصیلی programming languages مخصوص
- hosting/infrastructure کی MCP servers
- Claude کی internal architecture یا model weights
- Constitutional AI, RLHF, یا safety training
- Embeddings یا vector databases کی implementation details
- Computer use / browser automation
- Vision / image analysis
- Streaming API یا SSE
- Rate limiting اور detailed API cost calculations
- OAuth اور API key rotation details
- Cloud-provider-specific configs
- Performance benchmarks
- Prompt caching implementation details
- Token counting algorithms

## Preparation Recommendations

1. بنائیں agent ❌ ساتھ ایک Claude Agent SDK
2. کریں configure پر real project کو Claude Code
3. کریں test اور MCP tools design
4. بنائیں Data extraction pipeline
5. کریں Prompt engineering practice
6. سیکھیں Context management patterns

7. Escalation اور human-in-the-loop workflows سمجھیں

8. Practice exam حل کریں